MDPI

Article

Formalizing G-Code for Language Models: Grammar-Aware Hierarchical Tokenization of CNC Milling Toolpaths

Stephen S. Eacuello [1,*], Romesh S. Prasad [1] and Manbir S. Sodhi [1]

[1] Department of Industrial Systems Engineering, University of Rhode Island, Kingston, RI 02881, USA
* Correspondence: seacuello@uri.edu

Version June 16, 2026 submitted to Sensors

Abstract: G-code controls nearly all CNC manufacturing, yet standard tokenizers fail on it: 97% of a single workpiece's vocabulary entries are numeric coordinates, vocabulary scales with part geometry, and subword splitting destroys semantic units. These failures reflect a broader *Technical Language Processing* (TLP) challenge in which NLP tools trained on natural-language corpora break down on formally specified technical languages. We make three contributions. First, we formalize G-code toolpath syntax as a regular grammar and propose a three-level tokenization framework: a finite-state grammar enforcing type transitions and command-parameter associations, a hierarchical decomposition into type, identity, and value, and digit-by-digit numeric prediction that reduces the structural vocabulary to 19 tokens regardless of geometry. Second, we present a *design-space analysis* organized around three properties any practical G-code tokenizer should provide—bounded vocabulary, lossless coordinate representation, and value-level grammar validity—and show by case analysis over the three dominant tokenizer classes (atomic, subword, digit-decomposed) that only structured digit decomposition occupies the corner where all three hold simultaneously. The framework is therefore a principled choice among the standard families, not one design among many. Third, we verify every cell of the design-space table empirically. Across seven tokenizer families on 120 CNC milling files, only digit-decomposed and character-level tokenizers achieve near-zero held-out OOV ($\leq 0.002\%$); BPE families reach 0.025–4.81% and flat per-value collapses to 18–65%. Across five cross-validation folds and 552 test samples, decoders trained with the standard HuggingFace BPE pipeline (whitespace pre-tokenizer) reach $0.00 \pm 0.00\%$ free-generation grammar validity due to decoder-side whitespace reinsertion between subword pieces, while a byte-level BPE control that replaces the pre-tokenizer recovers to 97.1%—indistinguishable from the other non-fragmenting rows (96.98–98.31%). This separates the standard-pipeline failure (a decode-time artifact, fixable) from the merge-table-ambiguity concern (not observed empirically on this corpus). The KL divergence between grammar-masked and unmasked type-transition distributions is 0.005 ± 0.003 nats, two orders of magnitude below the regime where ASAp-style corrections become necessary. Under 5-fold cross-validation, the hierarchical decoder reaches 94.7% token accuracy and 84.4% sequence exact-match (test means), with structural predictions exceeding 96% and numeric digit prediction at 91.1%. A slim cross-entropy trainer used for the apples-to-apples cross-tokenizer experiments (Section 6.3) reaches a lower absolute accuracy; we report both regimes separately and do not conflate them.

Keywords: G-code; CNC machining; tokenization; language model; grammar constraints; hierarchical decoding; manufacturing

1. Introduction

Numerical control of machine tools dates to the early 1950s, when the Parsons Corporation and MIT's Servomechanisms Laboratory developed the first NC milling machine [1]. The programming language that emerged—G-code, standardized as RS-274 [2] and later ISO 6983 [3]—has since become the standard instruction set for CNC milling, turning, and multi-axis machining. Large language models now process Python, Java, SQL, and dozens of other languages [4–6], yet G-code remains comparatively underexplored relative to general-purpose programming languages, addressed directly by only a handful of recent efforts (Section 2).

The reason is structural: in a typical single-workpiece CNC milling vocabulary, 693 of 712 distinct vocabulary entries (97.3%) are numeric coordinates; only 19 cover commands, parameters, and control symbols. BPE [7] and WordPiece [8] split `X3.275` into `[X, 3, ., 275]`—destroying the semantic unit (Table 1; confirmed empirically in Table 13). Worse, vocabulary scales with geometry: every new workpiece introduces unseen coordinates and out-of-vocabulary failures. This is an instance of the broader *Technical Language Processing* (TLP) challenge [9]: standard NLP tools, trained on natural-language corpora, fail systematically on domain-specific technical languages whose vocabulary, syntax, and semantics diverge from general text.

Why tokenization of G-code matters now.

Fixing tokenization is a prerequisite for four downstream applications that have recently become active research targets in CNC monitoring, each of which breaks in a specific way when numeric coordinates are fragmented. (i) *Process verification* in regulated industries (aerospace, medical devices, defense) requires demonstrating that the intended toolpath, feed rate, and depth of cut actually occurred on the machine; classification-based monitoring [10–12] can confirm *what operation type* ran but not *which specific G-code line* was executed, and the upgrade from operation-level to line-level verification requires a decoder that can reconstruct G-code tokens from sensor windows—a task that fails immediately if the token stream cannot round-trip to a parseable line. (ii) *Manufacturing cybersecurity* detects subtle G-code tampering (coordinate perturbations, command substitutions, feed-rate modifications) that evade sensor-only anomaly detectors precisely because the machine executes the modified program faithfully and sensors report normal dynamics; comparing a claimed program against a sensor-reconstructed one creates an independent physical verification channel, which again demands grammatically valid reconstruction at the value level. (iii) *Cost-effective distributed retrofit monitoring* needs sensor selection criteria that go beyond operation classification to measure each modality's contribution to program-level reconstruction, which is only well-posed when the prediction target is a structured G-code sequence rather than a class label. (iv) *Digital twins and toolpath fingerprinting* treat the forward map (G-code $\rightarrow$ sensor stream) and its inverse (sensor stream $\rightarrow$ G-code) as dual views of the same physical process; an ill-formed inverse output—whitespace-corrupted subwords that no parser will accept—breaks the digital-twin closure property. All four applications share a common requirement: translation between symbolic program semantics (commands, addresses, numeric values) and continuous physical measurements must preserve syntactic validity at the parser level. That is the problem this paper addresses.

Our framework rests on three observations:

1. **The language is simple.** Despite hundreds of defined G-codes, real toolpaths use four motion commands (G0, G1, G2, G3). The per-line grammar is regular and capturable by a finite-state automaton (Sections 3–3.2).
2. **Structure and values are separable.** Selecting which command and which parameters is a small discrete problem; predicting coordinate values to micrometer precision is continuous. Each benefits from a different mechanism (Sections 5.1–5.2).
3. **Numbers are structured objects.** A coordinate is a composition of sign and digits, not an atomic symbol. Digit-by-digit prediction eliminates vocabulary explosion and generalizes to unseen geometry (Section 5.2).

Table 1. Subword Tokenization vs. Our Hybrid Tokenization (verified on actual tokenizers)

Method	Tokens
Input	`G2 X3.275 Y0.375 R0.083`
cl100k (GPT-4)	`G`, `2`, `␣X`, `3`, `.`, `275`, `␣Y`, `0`, `.`, `375`, `␣R`, `0`, `.`, `083` (14 tok.)
Domain BPE†	`G`, `2`, `X3`, `.`, `275`, `Y0`, `.`, `3`, `75`, `R0`, `.`, `0`, `8`, `3` (14 tok.)
Ours	`G2`, `X`, `NUM_X_3275`, `Y`, `NUM_Y_375`, `R`, `NUM_R_830` (7 tok.)

␣ denotes a leading space attached to the next token (tiktoken's convention).

†HuggingFace BPE trained on the corpus (vocab = 975, whitespace pre-tokenizer); verified on fold 1 checkpoint.

Both BPE variants fragment numeric coordinates, but in different places: cl100k keeps `275`, `375`, `083` intact while splitting on the address letter; Domain BPE merges address-digit pairs (`X3`, `Y0`, `R0`) but over-fragments the fractional digits (`3`, `75` in `375`; `0`, `8`, `3` in `083`). Neither yields a contiguous address-value word, which is the failure mode Definition 3 captures.

Figure A1 (Appendix C) provides an end-to-end overview. Specifically, we contribute:

1. A formal regular grammar for G-code toolpath lines, with five deterministic syntactic rules (Section 3.2).
2. A design-space characterization of G-code tokenizers: three properties (bounded vocabulary, lossless coordinates, grammar validity) and a case analysis over the three dominant tokenizer classes showing that only digit-level decomposition occupies the corner where all three properties hold simultaneously (Section 4). The remainder of the paper provides constructive and empirical support for that corner.
3. A hierarchical tokenization decomposing tokens into type, identity, and digit-level value—reducing structural vocabulary to 19 fixed tokens versus 712+ that scale with geometry (Sections 5.1–5.2).
4. A grammar-constrained decoding mechanism combining hard logit masks and soft training penalties, guaranteeing syntactic validity at $O(1)$ cost per step for the covered grammar fragment (Section 5.3).
5. Empirical validation on 120 G-code files from Autodesk Fusion 360 CAM running on a TinyG-controlled Bantam Tools Desktop CNC mill: a tokenizer-zoo evaluation populating every cell of the design-space table, plus 2.5× token reduction over GPT-4's BPE, 99.9% grammar coverage on the evaluation corpus, Heaps' law vocabulary scaling [13], and downstream decoder performance of 94.7% token accuracy with structural predictions exceeding 96%, plus an ablation showing the auxiliary structural-head losses are essential to stable flat-decoder training (94.9% vs. 75.0% without them) (Section 6).

Scope.

The grammar, precision configuration, and empirical evaluation in this paper cover a specific subset of RS-274D G-code: the four core motion commands (`G0`–`G3`) with five positional and feed-rate parameter addresses (`X`, `Y`, `Z`, `F`, `R`), plus five auxiliary literals for program setup and shutdown. Our 120-file dataset comes from one CAM system (Fusion 360), one firmware family (TinyG; we have also used grbl on the same hardware), and one machine class (a desktop-scale 3-axis mill with a $\sim$140×102×60 mm work envelope). Canned cycles (`G81`–`G89`), tool compensation (`G41/G42`), `I/J/K` arc-center format, full 5-axis programs, and controller-specific dialects (Fanuc, Siemens, Mazak, Haas) are outside the current grammar but extend naturally via additional lexical categories and DFA states (discussed in Sections 3.2 and 7). Claims about vocabulary growth, grammar coverage, and held-out OOV should be read in this scope; we discuss generalization to larger work envelopes and alternative precision configurations in Section 7. This paper addresses the *symbolic-representation* half

of sensor-based line-level verification—the tokenizer and grammar that make a sensor-driven G-code reconstruction parseable. The sensor pipeline itself (the multi-modal encoder and its modality-level analysis) is the subject of companion work; the present evaluation treats that encoder as a frozen, fixed conditioning source rather than a contribution, so the experiments here characterize tokenization and grammar, not sensing.

Table 2. Complete Structural Vocabulary (19 Tokens)

Category	Tokens	Count
Special	`PAD`, `BOS`, `EOS`, `UNK`, `MASK`	5
Command	`G0`, `G1`, `G2`, `G3`	4
Parameter	`X`, `Y`, `Z`, `F`, `R`	5
Auxiliary	`G53`, `M3`, `M5`, `M6`, `M30`	5
Total structural		19
Observed in dataset (Table 14)		7–9

Numeric values predicted by digit heads (Section 5.2), not vocabulary entries. `UNK`/`MASK`/`PAD` are reserved.

2. Related Work

Technical Language Processing. Brundage et al. [9] define TLP as the adaptation of NLP tools to technical domains where standard methods fail due to specialized vocabulary, non-standard syntax, and semantic divergence from general text. The core TLP research program addresses *human-written* technical prose. Hodkiewicz and Ho [14] show that maintenance work orders suffer from misspellings, inconsistent abbreviations, and missing fields; Sexton et al. [15] benchmark keyword extraction methods on these records; Sharp et al. [16] develop semi-autonomous tagging for unstructured maintenance logs. Subsequent work extends TLP to equipment reliability [17], fault diagnosis [18], and broader technical text adaptation [19]. In all these cases, the NLP failure mode is *lexical*—jargon, abbreviations, and domain-specific terminology that standard tokenizers and taggers do not recognize.

G-code presents a fundamentally different TLP challenge. It is not human-written prose but a *formally specified machine language* with deterministic syntax (Section 3.2). The failure mode is not lexical but *structural*: 97% of tokens are numeric coordinates whose vocabulary scales with geometry, and standard subword tokenizers destroy semantic units rather than misrecognizing them (Table 1). Custom dictionaries and entity taggers—the standard TLP toolkit—cannot address this. Our work extends TLP to formal technical languages, where adaptation requires formal grammars and structured numeric prediction rather than domain lexicons.

Table 3. TLP Challenges: Unstructured Prose vs. G-Code

	Prose TLP	G-Code (this work)
Source	Human-written	Machine-generated
Syntax	Free text	Deterministic grammar
Vocab issue	Jargon, abbrev.	Numeric explosion
NLP failure	Unknown terms	OOV coordinates
Solution	Custom dictionaries	Formal grammar + digit decomposition

LLMs for G-code and CNC. Jignasu et al. [20] evaluate six LLMs on G-code comprehension, debugging, and manipulation for additive manufacturing, finding wide performance variation—GPT-4 identifies errors readily, while smaller models fail even with additional prompting. GLLM [21] fine-tunes StarCoder-3B [6] for natural-language-to-G-code translation but retains BPE, which fragments coordinates destructively (Table 1). Image2Gcode [22] generates toolpaths via diffusion, bypassing tokenization entirely. ChatCNC [23] uses LLMs for CNC monitoring but consumes rather than generates G-code. Broader ML work in CNC targets anomaly detection [10], tool wear [11], and vibration analysis [12]—all consuming sensor data without modeling G-code as a language. None address tokenization; our framework is complementary to all of them.

Manufacturing security. A parallel literature documents cyber-physical attacks on digital manufacturing and their detection. Belikovetsky et al. [24] demonstrate a sabotage attack that weakens a printed part by covertly altering its toolpath, and Sturm et al. [25] show that malicious edits to design and toolpath files survive to the physical artifact. Defensive work detects such tampering from physical side channels—for example, Chhetri et al. [26] detect kinetic cyber-attacks by modeling the machine's acoustic and other analog emission side channels. These works are drawn from additive manufacturing, but the toolpath-tampering threat model and the side-channel verification principle carry over to subtractive CNC milling; a CNC tamper detector is out of scope here. This literature motivates application (ii) in Section 1: a sensor-reconstructed program offers an independent, physically grounded view of what the machine actually executed. The present paper supplies the symbolic-representation prerequisite for that view—a parseable, grammar-valid reconstruction target—and does not itself propose or evaluate a tamper detector.

Numeric tokenization. Subword methods fragment numbers destructively [27]. Singh and Strouse [28] show that tokenization strategy—particularly digit-level granularity and right-to-left ordering—substantially impacts arithmetic performance in frontier LLMs. Alternatives include xVal [29] (scalar multiplier on a `<NUM>` token), per-digit decomposition for mathematical reasoning [30], single-token embeddings [31], and regression losses [32]. Cognetta and Okazaki [33] formalize BPE and WordPiece as finite-state transducers, connecting subword tokenization to regular-language theory. We combine digit-level decomposition with domain-specific conditioning (parameter type, operation context) and grammar constraints. Unlike xVal, our digit heads attach to any encoder; unlike regression, categorical digits naturally express per-position uncertainty.

Design-space analyses in related areas. Proposition 1 is structurally similar in spirit—though not in formal rigor—to well-known design-space results in distributed systems and formal language theory. The CAP theorem [34,35] organizes distributed storage around three properties (consistency, availability, partition tolerance) and identifies named operating corners that real systems must choose between. Chomsky's hierarchy [36] organizes formal languages by containments that mirror the expressive power of their recognizers. We invoke these precedents only as framing: our trichotomy is a constructive case analysis over the three dominant tokenizer classes (atomic, subword, digit-decomposed), not a theorem over a formally defined tokenizer model, and we do not claim impossibility in the CAP or Fischer–Lynch–Paterson (FLP) [37] sense. What our analysis does establish is that each of the three standard classes deterministically fails at least one of the three properties in Definitions 1–3, whereas digit decomposition satisfies all three by construction. The empirical battery in Section 6 populates every cell of the resulting table and confirms the predicted ranking.

Grammar-constrained decoding. PICARD [38] constrains SQL to valid parse trees; Outlines [39] compiles regular expressions into FSMs; XGrammar [40] handles context-free grammars via pushdown automata. Park et al. [41] show naïve masking distorts distributions and propose ASAp; their follow-up [42] achieves 17.7× faster preprocessing. Geng et al. [43] extend the paradigm with input-dependent grammars; SynCode [44] precomputes DFA-based mask stores for efficient enforcement. G-code is *regular*, not context-free, so our constraints require no compilation and evaluate in $O(1)$. Moreover, they are *semantic*—"G0 forbids F", "G2 requires R"—not purely syntactic. Because most transitions are near-deterministic (PARAMETER → NUMERIC is mandatory),

we expected the distribution distortion Park et al. identify to be small. We confirm this empirically in Section 6.5: across 5 folds the mean KL divergence between the grammar-masked and unmasked type distributions is 0.005 ± 0.003 nats, and the unmasked model places only 0.14% of its probability mass on grammar-disallowed types—two orders of magnitude below the regimes where ASAp-style corrections are necessary.

3. Background and Formal Grammar

3.1. G-Code program structure

We ground our design in the structure of real programs. Our dataset comprises 120 G-code files from Autodesk Fusion 360 CAM for a Bantam Tools Desktop CNC mill, covering face milling, pocket milling, and adaptive clearing at three feed-rate settings.[1] The 120 files contain 62,403 toolpath lines, but only 2,248 of those lines are unique after canonicalization—a ratio of ∼28-to-1. This duplication is not an artifact of our preprocessing; it is a structural feature of CAM-generated G-code, and it has direct implications for tokenizer evaluation.

CAM postprocessors generate G-code by sweeping a tool along a parameterized geometric path—a contour, a pocket boundary, an adaptive clearing pattern. The same coordinate values appear repeatedly because the tool returns to the same fixture features (entry retracts to the same safe Z, periodic clearance moves to the same staging point, repeat passes at incremented depths). Adaptive clearing in particular emits long sequences of short linear cuts whose start and end coordinates differ from one another by less than the discriminative resolution of the tokenizer's quantization, so many lines collapse to identical canonical forms after rounding to the supported precision. Modal persistence (Section 3) compresses the duplicates further: many lines omit the explicit motion command and rely on the previous `G1`, so two lines that differ only in their leading command word fold to the same parameter list.

This 28-to-1 compression matters for tokenizer evaluation in two ways. First, it means a tokenizer's vocabulary growth (Experiment A) is dominated by the unique lines, not by the raw line count: a flat-per-value tokenizer that emits 4,435 distinct tokens on the 120 files would emit nominally up to 62,403 if every line were unique, so the realized growth is roughly 1/14 of the worst case. Second, it means that two lines that look identical after canonicalization are not necessarily identical at the sensor-aligned level: the same canonical `X1.4919 Y1.4848` can appear at many different points in the toolpath, paired with different sensor-window prefixes, and the cross-validation splits respect program boundaries rather than canonical-line boundaries to prevent trivial within-program memorization. We return to this point in Appendix B.

A typical program has three phases (Listing 1).

3.1.1. Preamble: Setup Codes

The preamble sets coordinate mode (G90/G91), units (G20/G21), plane (G17–G19), work coordinate system (G54–G59), tool, and spindle. These codes appear once and constitute 5–15 lines—a small, fixed vocabulary requiring no numeric decomposition.

3.1.2. Toolpath: The Four Core Motion Commands

The toolpath—the cutting portion of every CNC program—is built from remarkably few commands:

Auxiliary commands exist—notably `G4` (dwell)—but appear rarely in toolpath sections. Every toolpath, no matter how complex—3D contours, spiral pockets, adaptive clearing, five-axis

[1] Dataset and implementation available at https://github.com/stephenseacuello/gcode_fingerprinting.

Listing 1: Anatomy of a CNC milling program.

```
% -- PREAMBLE (setup) --
G90              ; absolute coordinates
G21              ; metric units (mm)
G17              ; XY plane selection
G54              ; work coordinate system
T1 M6            ; tool change
S12000 M3        ; spindle on, 12000 RPM
% -- TOOLPATH (cutting) --
G0 X0.0 Y0.0 Z1.0   ; rapid to start
G1 Z-0.5 F200       ; plunge cut
G1 X3.275 Y0.375    ; linear cut (G1 modal)
X3.291 Z0.0003      ; implicit G1 continues
G2 X3.5 Y0.7 R0.08 ; clockwise arc
G3 X3.2 Y1.0 R0.08 ; counter-clockwise arc
G0 Z1.0              ; retract
% -- POSTAMBLE (shutdown) --
G53 G0 Z0.0     ; machine home Z
M5              ; spindle off
M30             ; program end
```

Table 4. Core Motion Commands and Parameter Rules

		X,Y,Z	F	R	I,J,K
G0	Rapid (reposition)	○	×†	×	×
G1	Linear cut	○	○	×	×
G2	Clockwise arc	○	○	●	●
G3	Counter-clockwise arc	○	○	●	●

● = required, ○ = optional, × = forbidden.

G2/G3: exactly one of R *or* I/J/K (not both). Our grammar and decoder implement only the R form; R cannot encode a full circle or an arc $> 180°$, for which the I/J/K center format (out of scope here; Sections 1 and 7) is required.

†Some controllers accept F on G0 but ignore it; we treat a rapid as carrying no programmed feed (a canonicalization choice, not a universal rule). No canonical line in our 120-file corpus carries F on a G0, so the G0 → no-F mask never rejects a real line.

simultaneous machining—is ultimately a sequence of straight lines (G1) and circular arcs (G2/G3), connected by rapid repositioning moves (G0).

3.1.3. Modal Persistence

Motion commands are *modal*: once issued, they persist until overridden. In Listing 1, `X3.291 Z0.0003` carries no explicit command—it executes under the previously active `G1`. In our data, approximately 60% of toolpath lines omit the command word. The grammar must therefore allow sequences beginning directly with a PARAMETER token (Figure 1).

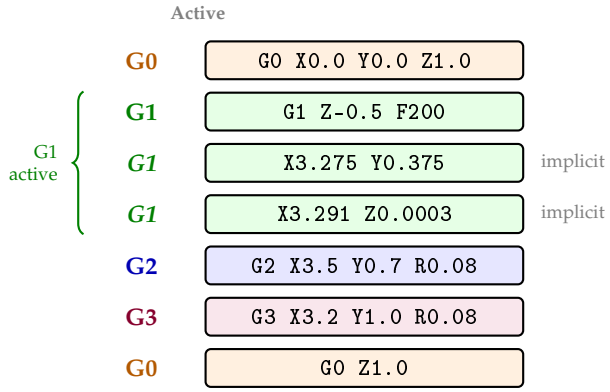

Figure 1. Modal state persistence. Lines 3–4 contain no command word; they execute under the previously active `G1`. The active modal command (shown left, italic for implicit) carries forward until a new motion command is issued.

3.1.4. Auxiliary Commands, Postamble, and Summary

Canned cycles (G81–G89) provide shorthand for drilling and boring; the controller firmware expands each into a deterministic G0/G1 sequence.[2] The *postamble* returns the machine to a safe state: spindle off (M5), coolant off (M9), home (G53 G0), program end (M30). Our tokenizer handles these as literal tokens without numeric decomposition. In our dataset, the toolpath dominates program content (80–90% of lines), with preamble (5–10%) and postamble (2–5%) as fixed overhead—justifying our focus on the four core motion commands rather than the full RS-274D standard.

3.2. Formal grammar

Given the structural regularity established in Section 3, we now formalize toolpath lines as lexical categories, syntactic rules, and a production grammar.

3.2.1. Lexical Categories

Four token types partition the vocabulary:

Table 5. G-Code Token Categories

Type	Category	Examples
0	SPECIAL	`PAD`, `BOS`, `EOS`
1	COMMAND	`G0`, `G1`, `G2`, `G3`
2	PARAMETER	`X`, `Y`, `Z`, `F`, `R`
3	NUMERIC	`NUM_X_3275`, `NUM_R_830`

[2] Tapping cycles (G84) additionally require spindle synchronization—a firmware operation outside the G0–G3 grammar.

We separate addresses from values as the key design choice. `X3.275` becomes two tokens: `X` (PARAMETER) encoding *which axis*, and `NUM_X_3275` (NUMERIC) encoding *how far*—fundamentally different information that benefits from independent prediction.

3.2.2. Syntactic Rules

Toolpath syntax follows five deterministic rules:

1. **Type Transitions.** COMMAND → PARAMETER/`EOS`; PARAMETER → NUMERIC; NUMERIC → PARAMETER/`EOS`. Commands and numerics never appear adjacent.
2. **Modal Group Exclusion.** At most one command from each modal group may appear per block. The motion group {`G0`, `G1`, `G2`, `G3`} is mutually exclusive.
3. **Command-Parameter Association.** Each command defines which parameters are valid, required, or forbidden (Table 4).
4. **Single-Letter Rule.** Each parameter address appears at most once per block. A line cannot contain two `X` values.
5. **Modal Persistence.** If no command is specified, the previously active modal command persists. A line containing only `X3.291 Z0.0003` executes under the last issued motion command.

RS-274D defines several modal groups (motion: G0–G3; plane: G17–G19; distance: G90/G91; units: G20/G21; coordinate system: G54–G59); commands within a group are mutually exclusive. Only the motion group matters during toolpath generation; the rest are set once in the preamble. Rules 1–4 apply within a single block. Rule 5 introduces cross-line state: the active modal command carries over from previous blocks. We treat each line independently, passing the modal command as external context.

3.2.3. Command-Parameter Associations and Execution Order

Parameter associations per the RS-274D standard [2] and the NIST interpreter specification [45] are formalized in Table 4. Our framework implements X, Y, Z, F, and R; additional addresses (I, J, K, S, P) can be added per controller dialect with appropriate precision settings (Table 8). Forbidden entries (×) become $-\infty$ logit masks; required entries (•) become positive biases (Section 5.3). The standard specifies a 21-step execution order within each block [45], with motion (G0–G3) at step 20 and stop codes last; our grammar does not model intra-block ordering.

3.2.4. Formal Grammar Definition

We define the per-line G-code grammar as a right-linear grammar $\mathcal{G} = (\Sigma, V, S, P)$ [46]:

Terminal alphabet. $\Sigma = \{\texttt{BOS}, \texttt{EOS}, \texttt{G0}, \texttt{G1}, \texttt{G2}, \texttt{G3}, \texttt{X}, \texttt{Y}, \texttt{Z}, \texttt{F}, \texttt{R}, \texttt{NUM}\}$

Nonterminals. $V = \{S, \mathit{Body}, \mathit{Cmd}, \mathit{Params}, \mathit{Param}, \mathit{Addr}\}$

Start symbol. S

Productions.

$$
\begin{aligned}
S &\to \texttt{BOS}\ \mathit{Body}\ \texttt{EOS} \\
\mathit{Body} &\to \mathit{Cmd}\ \mathit{Params} \mid \mathit{Params} \\
\mathit{Cmd} &\to \texttt{G0} \mid \texttt{G1} \mid \texttt{G2} \mid \texttt{G3} \\
\mathit{Params} &\to \mathit{Addr}\ \texttt{NUM}\ \mathit{Params} \mid \varepsilon \\
\mathit{Addr} &\to \texttt{X} \mid \texttt{Y} \mid \texttt{Z} \mid \texttt{F} \mid \texttt{R}
\end{aligned}
$$

Every production is right-linear (at most one nonterminal, rightmost), so $\mathcal{G}$ generates a regular language [46]. The *Addr* production shows the five currently implemented addresses; additional addresses (I, J, K, S, P) extend Σ without changing the grammar's regular structure. The grammar is recognized by a five-state deterministic finite automaton whose state table, transition function,

soundness and completeness arguments, and cross-line modal extension we defer to Appendix A; what matters here is that the constraint mask M used by Section 5.3 implements δ directly, with $M(q_i, t) = -\infty$ whenever $\delta(q_i, t)$ is undefined, and that mask lookup is $O(1)$ since $|Q| = 5$ and $|\Sigma| = 12$.

The grammar defines 19 structural terminals: 5 special (`PAD`, `BOS`, `EOS`, `UNK`, `MASK`), 4 commands (`G0`–`G3`), 5 parameters (`X`, `Y`, `Z`, `F`, `R`), and 5 auxiliary literals (`G53`, `M3`, `M5`, `M6`, `M30`). Of these, 7–9 appear in any given dataset (Table 14); the remainder are reserved for future extensions. Numeric values are predicted by digit heads (Section 5.2), not vocabulary entries. As a worked example, consider generating `G2 X3.5 Y0.7 R0.08` under the type-transition FSM of Figure 4: after PARAMETER `X`, only NUMERIC is valid; after the numeric token, `X` is masked (already used) and `R` is boosted (required for G2). Each constraint evaluates in $O(1)$.

4. A Design-Space Analysis of G-Code Tokenizers

Sections 3–3.2 establish what G-code is. We now analyze the space of possible tokenizers around three properties any practical G-code tokenizer should provide, and show by case analysis over the three dominant tokenizer classes (atomic, subword, digit-decomposed) that only the third class satisfies all three properties simultaneously. We do not claim an impossibility theorem in the formal sense of CAP or FLP: the case analysis is over the classes that dominate current practice, not over a formally defined space of all conceivable tokenizers, and the argument is constructive in that it names what each class gives up. Section 6 verifies each cell of Table 7 empirically.

4.1. Three properties

We define three properties any practical G-code tokenizer should provide. Each definition isolates a single failure mode of standard tokenization and is given a precise, testable form so that Section 6 can populate it with a real measurement.

Definition 1 (Bounded vocabulary)**.** *A tokenizer T has* bounded vocabulary *if there exists a constant K, independent of geometry, such that for every G-code corpus $\mathcal{C}$ at fixed precision p,*

$$|V_T(\mathcal{C})| \;\leq\; K,$$

where $V_T(\mathcal{C})$ denotes the set of distinct token types T emits on $\mathcal{C}$.

Definition 2 (Lossless coordinates)**.** *A tokenizer T is* lossless on coordinates *if for every numeric value v at supported precision p, the encode–decode roundtrip is exact:*

$$\mathrm{dec}_T(\mathrm{enc}_T(v)) \;=\; v,$$

equivalently $\sup_v |\mathrm{dec}_T(\mathrm{enc}_T(v)) - v| = 0$.

Definition 3 (Grammar validity)**.** *A tokenizer T* supports grammar validity *if both of the following hold:*

(i) every line $\ell \in L(\mathcal{G})$ admits an unambiguous encoding $\mathrm{enc}_T(\ell) \in V_T^*$*;*
(ii) the per-step output distribution under T can be restricted—via logit masking or an equivalent constraint mechanism—to the set $L(\mathcal{G})$ without removing any line in $L(\mathcal{G})$.

Property (ii) is non-trivial. A tokenizer that maps every coordinate to a single ⟨NUM⟩ token (e.g. xVal [29]) admits encodings, but the value-level distribution cannot be grammar-constrained: the constraint mechanism has no token at which to apply the mask. Subword tokenizers fail property (i): the same numeric value may decompose differently in different contexts, so the encoding is not unambiguous at the value level.

4.2. A structural characterization

The three properties of Section 4.1 look modest in isolation. We now show by case analysis over the three dominant tokenizer classes that only one class admits all three properties simultaneously.

Proposition 1 (Tokenizer trichotomy over dominant classes)**.** *Let T be a tokenizer for G-code coordinates at fixed precision p over an unbounded geometric domain, drawn from one of the three standard classes: atomic (one token per coordinate value), subword (BPE- or WordPiece-style merges over the coordinate string), or digit-decomposed (each value emitted as an address letter followed by a fixed-length digit sequence). Among these three classes, only digit-decomposed tokenizers with a finite sub-vocabulary closed under composition (e.g., decimal digits with sign) satisfy Definitions 1, 2, and 3 simultaneously.*

Case analysis. We argue by case over the three named classes of T. For each failing class we name the definition it violates and the structural reason.

Case 1: atomic tokenization. Suppose T treats each coordinate as a single token. The set of quantized coordinate values at precision p over an unbounded domain is countably infinite, so T must either collapse infinitely many values onto finitely many tokens (violating Definition 2) or preserve every value atomically and let $|V_T|$ scale with the observed geometric range (violating Definition 1). Either branch fails.

Case 2: subword tokenization. BPE and WordPiece fragment numeric strings into merge-derived subword pieces. The fragmentation has two distinct failure modes that we separate empirically in Section 6.3. *Sub-case 2a (decode-side reinsertion):* standard HuggingFace BPE pipelines use a whitespace pre-tokenizer whose inverse inserts whitespace between pieces on decode, so an in-vocabulary value like `0.1755` decodes to `"0 . 1755"` and is no longer parseable as a G-code word. This fails Definition 3 property (i) immediately, and empirically reaches $0.00 \pm 0.00\%$ free-generation grammar validity (Table 19). *Sub-case 2b (value-level constraint boundary):* even with a byte-level decoder that preserves the contiguous numeric string, the merge table is data-driven and context-dependent; identical values may decompose differently across contexts, and there is no consistent token slot that corresponds to a single coordinate value. This leaves Definition 3 property (ii) unsatisfied at the value level: the constraint mechanism has no semantically meaningful slot at which to enforce per-value rules (e.g., "if this is an arc parameter, mask values outside $[0, R_{\max}]$"). Subword tokenization can be made to pass property (i) by switching the decoder configuration—as the byte-level BPE row of Table 19 demonstrates—but it does not pass Definition 1 (active vocabulary grows with geometry) or the value-level form of property (ii) simultaneously.

Case 3: structured decomposition. Among the three classes, only digit decomposition encodes each value as a finite alphabet that composes back to any value at precision p. The smallest such alphabet is the set of decimal digits together with a sign symbol, an 11-element sub-vocabulary closed under composition. Definition 1 holds because the alphabet is constant in $|\mathcal{C}|$. Definition 2 holds because digit composition is exact at precision p. Definition 3 holds because each digit position is itself a discrete prediction slot that admits per-position masking. □

Scope of the case analysis.

Proposition 1 covers the three classes that dominate current NLP practice (atomic, subword, digit-decomposed) and character-level tokenization as a degenerate sub-case of decomposition over the ASCII alphabet (evaluated empirically in Section 6). It does not cover every conceivable tokenizer design: learned discrete codebooks, hash-based tokenization, and continuous scalar heads [29] each require a separate argument, which we address briefly in the remarks below. The claim is therefore a *characterization over the standard families*, not a universal impossibility result; the constructive contribution is naming what each class gives up rather than proving any class cannot exist. In particular, the byte-level BPE result of Section 6.3 does not weaken the proposition: it shows that the subword

class can be made to satisfy property (i) by changing the decoder configuration, while the discriminator that survives is Definition 1 (active vocabulary still grows with geometry) together with the value-level form of property (ii), which the fused-merge analysis of Proposition 2 shows subword tokenizers continue to fail regardless of decoder.

Proposition 2 (Fused-subword corollary)**.** *Let T be a subword tokenizer whose learned merge table produces at least one token whose surface form fuses an address letter with one or more numeric digits (e.g., a single token spanning `X3`, `Y0`, or `R0`). Then T does not admit a consistent token-position slot at which a per-value constraint mask can be applied, and therefore fails Definition 3 property (ii) at the value level, regardless of whether its decoder preserves numeric contiguity.*

Proof sketch. A per-value constraint mask (e.g., "if this is an arc radius, mask values outside $[R_{\min}, R_{\max}]$") must be evaluated at a position where the decoder is choosing a coordinate value, conditioned on already having committed to an address. Fused address–digit tokens collapse the address commitment and (part of) the value into a single token choice, so the mask would have to condition on the identity of the token currently being chosen rather than on a previously emitted address—which is not what value-level constraints do in any standard grammar-constrained decoding framework [38–40]. A consistent slot would require either (a) refusing to learn any fused address–digit merge, which is not a property enforced by BPE or WordPiece training, or (b) post-hoc splitting the merge at inference, which requires tokenizer-specific logic and breaks the $O(1)$ mask-lookup property. The byte-level BPE row of Table 19 empirically produces fused merges of this form (Table 1 shows `X3`, `Y0`, `R0`), so even though it passes property (i), it fails property (ii) at the value level. □

Proposition 2 formalizes what the byte-level BPE result empirically shows: the grammar-validity gap between BPE configurations is a property-(i) issue solved by the decoder configuration, but the grammar-*constrainability* gap between BPE and digit decomposition is a property-(ii) issue that survives any decoder configuration, because the underlying merge table has already collapsed the address–value boundary.

Remarks on the case analysis.

The characterization is robust to several variations. *Variable per-parameter precision* (Table 8) does not change the argument: each address a gets its own digit-position count n_a, the operative alphabet is still the eleven decimal digits plus sign, and the decomposed sub-vocabulary remains constant in $|\mathcal{C}|$. *Mixed atomic-and-decomposed hybrids* inherit the worst failure of each component: any atomically encoded address still has unbounded geometric range and fails Definition 1. *Structured tokenizers with reserved literals* (such as ours, which adds 19 fixed literals to an 11-element digit alphabet) preserve Definition 1 because the literals are constant in $|\mathcal{C}|$.

Two structural classes lie outside the three named classes of Proposition 1 and warrant separate comment. *Continuous-valued tokenizers* such as xVal [29] replace the value-level token with a scalar head; Definition 3 property (ii) fails by inspection because the constraint mechanism has no discrete token slot at which to apply a value-level mask. *Learned discrete codebooks* (e.g., VQ-style value quantizers) can in principle occupy the all-checkmarks corner if the codebook is closed under composition, but we do not evaluate them in this work. The unquantized limit $p \to 0$ has an uncountable value space, so Definition 1 fails for every token-based tokenizer; our reliance on a fixed positive p is a necessary precondition for the all-checkmarks corner to exist at all.

Proposition 1 characterizes the framework's design choices as the natural corner for current tokenizer practice: among the three classes standard NLP systems actually use, digit decomposition is the one that occupies the (BOUNDED, LOSSLESS, GRAMMAR) corner simultaneously, and the grammar-constrained decoding of Section 5.3 is what activates property (ii) of Definition 3.

Section 6.3 documents both sub-failures of Case 2 empirically. Standard whitespace-pre-tokenized BPE reaches $0.00 \pm 0.00\%$ free-generation grammar validity across 5 folds and 552 test samples because

of decoder whitespace reinsertion (sub-case 2a); a byte-level BPE control that avoids that artifact is included to isolate the merge-table ambiguity (sub-case 2b) from the decoder configuration.

4.3. Worked example: tokenizing R0.1755

To make the trichotomy of Proposition 1 concrete, we trace what each of the three structural cases does on a single G-code parameter, the arc-radius value `R0.1755`. We pick `R` (rather than `X` or `Y`) so that the underlying value is exactly representable at the supported precision: in our implementation `R` uses precision $p = 0.0001$ mm (Table 8), so 0.1755 quantizes to bucket 1755 without rounding error. Each row of Table 6 represents one tokenizer occupying one of the three structural cases, encodes `R0.1755`, and reports which definitions it satisfies.

Table 6. Worked Example: `R0.1755` Through One Tokenizer per Structural Case

Tokenizer	Encoded form	Def. 1	Def. 2	Def. 3	Failure mode
Case 1: Atomic Flat per-value	`[R0.1755]` (1 token)	×	✓	✓	Vocabulary scales with geometry: every new arc radius adds a new token; held-out OOV reaches 18–65% (Table 18).
Case 2: Subword Domain BPE	`[R, 0, ., 17, 55]` (5 tokens)	✓	∼	×	Decoder reinserts whitespace at every BPE boundary, producing `"R0 . 1755"`; the parser cannot recover the address–value pair, even though every subword is correct.
Case 3: Decomposed Hierarchical (ours)	`[R, SIGN+, D1, D7, D5, D5]` (6 tokens)	✓	✓	✓	None: the address letter `R`, sign, and four digit positions compose back to the integer bucket 1755, which decodes to the value $1755 \times 0.0001 = 0.1755$ mm exactly, and each prediction slot admits per-position grammar masking.

All three rows encode the same physical quantity. The differences are not in what the tokenizer can in principle represent but in which definition each one structurally fails: Case 1 makes vocabulary unbounded, Case 2 makes the value-to-string round trip ambiguous, and Case 3 is the only configuration in which all three definitions hold simultaneously.

The Case 2 row deserves a closer look. Domain BPE tokenizes `R0.1755` into the five subword pieces `[R, 0, ., 17, 55]`, all of which are perfectly retrievable, and a trained decoder under this tokenizer routinely predicts the right subword sequence in our experiments. The failure happens at decoding time. The HuggingFace BPE decoder inserts whitespace between subword pieces from a Whitespace pre-tokenizer, so the reassembly emits `"R0 . 1755"` instead of `"R0.1755"`. The address letter is now separated from its value by spaces, and the regex-based G-code parser cannot recognize the result as a parameter–value pair. This is property (i) of Definition 3 failing in practice, and it explains the $0.00 \pm 0.00\%$ free-generation grammar validity reported for every BPE row of Table 19 across five folds and 552 test samples. The exact fragmentation is value-dependent: for radii such

as `R0.083` the same merge table instead fuses the address letter into an `R0` token (Table 1), the fused address–value merge that Proposition 2 shows blocks value-level constraints regardless of the decoder.

We chose `R` for this example because 0.1755 mm is exactly representable at `R`'s native precision of $p = 0.0001$ mm. Coarser-precision parameters (`X`, `Y`, `Z` at $p = 0.001$ mm; Table 8) would incur a quantization error bounded by $p/2$ on the same value. Experiment B confirms this empirically: the mean reconstruction error on `X`, `Y`, and `Z` is approximately 0.25 μm, exactly half the 1 μm quantization step.

4.4. Tokenizer families and the design space

Table 7 maps every plausible tokenizer family onto the three axes. Each entry is verified empirically in Section 6: vocabulary growth (Experiment A) populates the BOUNDED column, coordinate reconstruction error (Experiment B) populates the LOSSLESS column, free-generation grammar validity (Experiment C) populates the GRAMMAR column, and held-out OOV rate (Experiment D) confirms that vocabulary boundedness is a measurable failure mode rather than a definitional artifact.

Table 7. Design-Space Coverage of G-Code Tokenizer Families

Tokenizer family	**Bounded V**	**Lossless**	**Grammar**
GPT-4 BPE (cl100k)	✓	×	$\sim^{**}$
GPT-2 BPE (r50k)	✓	×	$\sim^{**}$
Domain BPE (whitespace)	✓	×	$\times^{*}$
Domain BPE (byte-level)	✓	×	$\sim^{**}$
WordPiece	✓	×	$\times^{*}$
Flat per-value	×	✓	✓
Character-level	✓	✓	$\times^{***}$
xVal scalar [29]$^{\#}$	✓	$\sim^{\dagger}$	×
Regression head$^{\#}$	✓	$\sim^{\dagger}$	$\sim^{\ddagger}$
Hierarchical + digits (ours)	✓	✓	✓

*Fails Def. 3 via sub-case 2a (decoder-side whitespace reinsertion); empirical grammar validity $0.00 \pm 0.00\%$ (Table 19).

**Passes property (i) when the decode path preserves numeric contiguity (byte-level BPE empirically at 97%; GPT-4 compact at 97%), but still fails the value-level form of property (ii) because the merge table exposes no consistent token boundary for address–value pairs.

***Char-level FSAs over digit strings explode combinatorially before grammar productions can attach.

†Bounded by float precision, not exact at quantized precision p.

‡Constrainable at the address level but not at the value level.

$^{\#}$Theoretical placement based on published behavior; not evaluated empirically in this work. The other rows are trained and measured on our 120-file corpus (Section 6).

The proposition predicts the table; the table predicts the experiments; the experiments verify the table. The remainder of the paper presents the constructive instantiation of the bottom row and the empirical support for the rest.

Figure 2 renders Table 7 as a three-axis cube. Each corner is a (BOUNDED, LOSSLESS, GRAMMAR) Boolean tuple, and each tokenizer family lives at the corner that matches its row in the table. Five of the eight corners are occupied; three are empty (e.g. (BOUNDED, LOSSLESS, ¬GRAMMAR) in our taxonomy contains only character-level, and (¬BOUNDED, ¬LOSSLESS, GRAMMAR) is structurally empty under our definitions). Proposition 1 is the statement that exactly one corner—(BOUNDED, LOSSLESS, GRAMMAR)—contains the hierarchical decomposition family and that every other tokenizer architecture must accept at least one ¬ in its tuple.

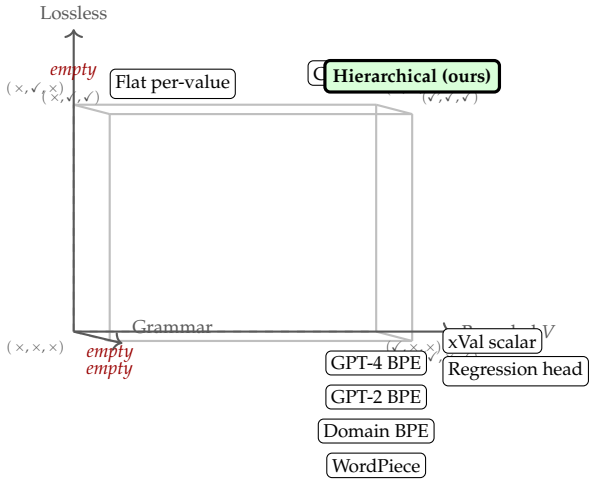

Figure 2. The (BOUNDED, LOSSLESS, GRAMMAR) design space as a three-axis Boolean cube. Each axis is one of the three properties of Definitions 1–3; each corner is a possible combination; each tokenizer family lives at the corner whose Boolean tuple matches its row in Table 7. Proposition 1 is the statement that the front-top-right corner $(\checkmark,\checkmark,\checkmark)$ is reachable only by the structured-decomposition family (Case 3 of the proof), and the dashed cube edges denote unreachable transitions: no incremental modification of an atomic or subword tokenizer can move it into the all-checkmarks corner without changing its structural class.

5. Method

This section describes the instantiation of the digit-decomposition corner of Table 7: the canonicalization and hybrid tokenization step (Section 5.1), the hierarchical token decomposition that occupies Case 3 of Proposition 1 (Section 5.2), and the grammar-constrained decoding mechanism that activates property (ii) of Definition 3 (Section 5.3).

5.1. Tokenization scheme

5.1.1. Canonicalization

Raw G-code undergoes five normalization steps to eliminate surface variation across CAM systems: (1) strip comments (`;` and `(...)`), (2) strip line numbers (`N`-codes) and checksums, (3) uppercase, (4) normalize leading zeros on command codes (`G01` → `G1`), and (5) collapse whitespace. For example, `n20 g02 x3.275 r0.083 ; arc` canonicalizes to `G2 X3.275 R0.083`.

Variable-precision coordinate strings.

Step 4 is the only normalization step that touches numeric values, and it applies only to command leading zeros; trailing-zero differences on coordinate strings (`X3.2` vs. `X3.2000`) are *not* resolved at canonicalization. They are made equivalent at the next stage: quantization (below) maps both to the same integer bucket at the parameter's precision (in this case, $\text{round}(3.2/0.001) = \text{round}(3.2000/0.001) = 3200$, emitted as `NUM_X_3200`). The tokenizer is therefore firmware-agnostic: TinyG (the controller on our Bantam Tools Desktop CNC), grbl, and other firmwares that interpret trailing zeros differently in their parsers all receive identical tokens from our preprocessing, because the distinction is absorbed by bucket-level quantization before any model ever sees a coordinate.

5.1.2. Hybrid Tokenization

We use a *hybrid* strategy. Literal mode creates one token per coordinate value (vocabulary explosion). Split mode over-fragments commands (`G1` → `G`, `1`). Hybrid mode preserves command semantics while separating addresses from values:

```
G2 X3.275 Y0.375 R0.083
-> [BOS, G2, X, NUM_X_3275, Y,
    NUM_Y_375, R, NUM_R_830, EOS]
```

5.1.3. Quantization

We quantize numeric values at domain-appropriate precision, mapping a float value v for parameter a to an integer bucket:

$$\text{bucket}(v, a) = \text{round}\left(\frac{v}{\text{precision}(a)}\right) \tag{1}$$

For example, `X3.275` $\rightarrow \text{round}(3.275/0.001) = 3275 \rightarrow$ `NUM_X_3275`; `R0.083` $\rightarrow \text{round}(0.083/0.0001) = 830 \rightarrow$ `NUM_R_830`.

Table 8 combines quantization precision, value ranges, and digit-head configuration for each parameter.

Table 8. Parameter Precision, Ranges, and Digit Configuration

Param	Prec.	Range	Int.	Dec.
X, Y	0.001 mm	±500 mm	2	4
Z	0.001 mm	−200–50 mm	1	4
I, J	0.0001 mm	±500 mm	1	4
R	0.0001 mm	0.001–500 mm	1	4
F	1 mm/min	0.1–15000	2	1
S	10 RPM	0–30000	5	0

Int. = max integer digits, Dec. = decimal digits. These define digit-head dimensionality per parameter.

5.1.4. The Vocabulary Explosion Problem

Table 9 quantifies the problem for a single workpiece. Each new part compounds OOV failures.

Table 9. Vocabulary Size vs. Precision

Strategy	Size	OOV	% Numeric
2-digit bucketed	170	∼0%	87%
Full precision	712	0%	97.3%
Digit-by-digit	19	0%	0%[†]

[†]Numeric values predicted by digit heads, not vocabulary.

Figure 3 plots measured vocabulary growth across our 120-file dataset (Section 6): flat vocabulary reaches 4,435 tokens while structural vocabulary stays at 9 (19-slot capacity).

5.1.5. End-to-End Pipeline

Figure A1 (Appendix C) traces a single G-code line through the complete pipeline: canonicalization, hybrid tokenization, type classification, hierarchical decomposition, and digit-by-digit prediction.

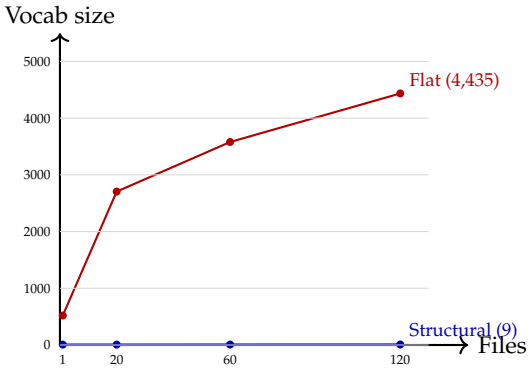

Figure 3. Measured vocabulary growth across 120 G-code files. Flat vocabulary grows from 520 to 4,435 tokens; structural vocabulary remains fixed at 7–9 tokens. The numeric fraction rises from 97.3% (single part, Table 9) to 99.8% across 120 files. Data from Table 14.

5.2. Hierarchical token decomposition

5.2.1. From Flat to Structured Prediction

A flat vocabulary selects from $|\mathcal{V}|$ classes with no structural bias—the model must independently learn that `NUM_X_3275` and `NUM_X_3276` are numerically adjacent. Our hierarchical approach decomposes prediction into a cascade:

$$P(\text{token}) = P(\tau) \cdot P(\iota \mid \tau) \cdot P(\mathbf{v} \mid \iota, \tau) \tag{2}$$

where τ is the token type, ι is the identity (command or parameter address), and $\mathbf{v}$ is the numeric value. This factorization follows the class-based language model tradition [47,48]: tokens are grouped into classes (types), and prediction proceeds hierarchically through class then member. Our contribution is not the factorization itself but its instantiation for G-code: the classes are grammar-derived (not learned), the third level decomposes into digits rather than selecting from a fixed set, and the entire cascade integrates with the DFA-based grammar constraints from Section 3.2.4. Figure A2 (Appendix C) illustrates this cascade with a concrete example.

5.2.2. Level 1: Token Type (4 classes)

A four-way classifier determines the token's role—the "part of speech" of G-code. This prediction is nearly deterministic given the grammar: after PARAMETER, the type *must* be NUMERIC.

5.2.3. Level 2: Identity Prediction

Conditioned on token type:

- COMMAND → {G0, G1, G2, G3} (4 classes)
- PARAMETER → {X, Y, Z, F, R, ...} (5+ classes)
- SPECIAL → {PAD, BOS, EOS, UNK, MASK} (5 classes)
- NUMERIC → proceed to Level 3

5.2.4. Level 3: Digit-by-Digit Numeric Prediction

Instead of predicting a numeric token as one of hundreds of vocabulary entries, we decompose each value into structured components:

$$v = s \cdot \left(\sum_{i=0}^{1} d_i \cdot 10^{1-i} + \sum_{j=0}^{3} d_{2+j} \cdot 10^{-(j+1)} \right) \tag{3}$$

where $s \in \{+1, -1, 0\}$ is the sign and $d_0, \ldots, d_5 \in \{0, \ldots, 9\}$ are the six digits in the format $\pm d_0 d_1 . d_2 d_3 d_4 d_5$.

Table 10. Digit-by-Digit Decomposition Examples

Token	Val.	Sign	Digits	Format
NUM_X_3291	3.291	+	$[0, 3, 2, 9, 1, 0]$	+03.2910
NUM_Y_375	0.375	+	$[0, 0, 3, 7, 5, 0]$	+00.3750
NUM_Z_n43	−0.043	−	$[0, 0, 0, 4, 3, 0]$	−00.0430
NUM_R_830	0.083	+	$[0, 0, 0, 8, 3, 0]$	+00.0830

Note: The 6-digit format applies to positional parameters. Feed rate (F) uses fewer digits ($d_0 d_1 . d_2$). The digit head is parameter-aware: each address specifies its own digit count to match its physical resolution.

This decomposition provides five advantages: (1) **zero OOV**—any value is representable; (2) **numeric proximity**—3.275 and 3.276 differ in one digit; (3) **compositionality**—each digit is a 10-class problem; (4) **precision control**—append digits without vocabulary growth; (5) **generalization**—works for any geometry without retraining.

Why digits, not regression? A regression head (like our auxiliary $\hat{v}$) produces a point estimate, losing the ability to express structured uncertainty—e.g., "confident the integer part is 3, uncertain whether the fourth decimal is 2 or 3." Digit heads produce a joint distribution over the value space and integrate with grammar-based logit masking, which has no analogue for continuous outputs. xVal [29] takes the regression approach, preserving magnitude but losing both digit-level structure and constraint compatibility.

Limitation: digit independence. The six heads predict independently. To characterize the cost of this assumption, we compute the mutual information (MI) and its normalized form (normalized mutual information, NMI) between adjacent digit positions across 13,595 numeric values in our dataset:

Pair	d_0-d_1	d_1-d_2	d_2-d_3	d_3-d_4
MI (bits)	0.01	0.85	0.45	0.09
NMI	0.86	0.87	0.17	0.03

The integer-decimal boundary (d_1-d_2) exhibits strong dependency (0.85 bits), as expected: knowing the integer part constrains the decimal range. Later decimal positions (d_3-d_5) are near-independent (MI < 0.2 bits, NMI < 0.06). An autoregressive chain predicting $d_0 \to d_1 \to d_2$ then independent d_3-d_5 could capture the high-MI boundary while keeping most predictions parallel.

5.2.5. Conditioning Signals

Digit prediction is conditioned on the **parameter type** (X, Y, Z, F, R—determining physical quantity and range) and **operation type** (face, pocket, adaptive—influencing typical value distributions). These embeddings are concatenated with the hidden state and projected before digit prediction.

5.2.6. Training Objective

The hierarchical decomposition yields a multi-head loss:

$$\mathcal{L} = \mathcal{L}_\tau + \mathcal{L}_\iota + \mathcal{L}_s + \sum_{i=0}^{5} \mathcal{L}_{d_i} + \lambda_{\text{aux}} \mathcal{L}_{\hat{v}} + \mathcal{L}_{\text{grammar}} \tag{4}$$

where $\mathcal{L}_\tau$, $\mathcal{L}_\iota$, $\mathcal{L}_s$, $\mathcal{L}_{d_i}$ are cross-entropy losses on type (4), identity, sign (3), and digit (10) predictions; $\mathcal{L}_{\hat{v}}$ is Huber loss on an auxiliary scalar prediction ($\lambda_{\text{aux}} = 0.1$); and $\mathcal{L}_{\text{grammar}}$ sums soft constraint penalties (Section 5.3). Losses are masked to apply only at positions of the corresponding type.

5.2.7. Token Reassembly

Given predictions from all levels, a G-code token is reassembled deterministically:

function REASSEMBLE(τ, ι, s, **d**)
  if $\tau \in \{\text{Spcl}, \text{Cmd}, \text{Par}\}$ **then**
    return lookup$[\tau, \iota]$ ▷ e.g. `G2`, `X`, `EOS`
  else if $\tau = \text{Num}$ **then**
    $v \leftarrow \sum_i d_i \cdot 10^{1-i}$ ▷ compose digits
    if $s = -$ **then** $v \leftarrow -v$
    $a \leftarrow$ last_param_address
    return `NUM_{`a`}_{round(`v`/prec(`a`))}`
  end if
end function

This closes the loop: hierarchical predictions → digits → G-code token → valid line.

Inference-time overhead.

The hierarchical head cascade replaces a single $d_{\text{model}} \to 712$ softmax with nine smaller projections ($d_{\text{model}} \to \{4, 14, 3\}$ plus six $d_{\text{model}} \to 10$ digit heads). On a CPU micro-benchmark ($d_{\text{model}} = 384$, single-sample forward pass, 1000-iteration mean), the flat head costs $\approx 38\,\mu$s/token and the hierarchical cascade costs $\approx 73\,\mu$s/token—a $+35\,\mu$s overhead per token, roughly $+93\%$ over the flat head. In absolute terms this is still well below real-time CNC monitoring latency budgets: at 50 Hz sensor sampling (a 20 ms budget per window), a single token costs $\approx 73\,\mu$s ($\sim 270\times$ headroom) and decoding a full 7–9-token line costs ≈ 0.5–0.66 ms (~ 30–$40\times$ headroom), before adding the sensor-encoder forward pass. The overhead is dominated by kernel launch cost on small tensors; on GPU or with batched inference the relative cost shrinks because the heads can run in parallel. We therefore treat the cascade overhead as negligible for monitoring applications and note it only for completeness.

5.3. Grammar-constrained decoding

We enforce the grammar via hard constraints (inference-time logit masking) and soft constraints (training-time penalties).

5.3.1. Hard Constraints via Logit Masking

The logit mask implements the DFA transition function from Section 3.2.4. Define $M : Q \times \Sigma \to \{0, -\infty\}$ as:

$$M(q, t) = \begin{cases} 0 & \text{if } \delta(q, t) \text{ is defined} \\ -\infty & \text{otherwise} \end{cases} \tag{5}$$

At each decoding step, the masked logits are $\tilde{\ell}^{(t)} = \ell^{(t)} + M(q_t, \cdot)$ where q_t is the current DFA state. By construction, the mask is **sound**: every generated sequence is in $L(\mathcal{A}) = L(\mathcal{G})$, since only transitions defined in δ have non-zero probability. It is also **complete**: every string in $L(\mathcal{G})$ can be generated, since the mask never blocks a valid transition.

Table 11 shows the mask in tabular form.

Table 11. Type Transition Mask (✓ = valid, × = masked to $-\infty$)

Prev.	SPCL	CMD	PAR	NUM
`BOS`	×	✓	✓	×
COMMAND	EOS	×	✓	×
PARAMETER	×	×	×	✓
NUMERIC	EOS	×	✓	×

Beyond type transitions, command-specific rules further constrain parameter selection:

- **G0 → no F.** Rapid traversals have no programmed feed rate; the `F` logit is masked to $-\infty$.
- **G2/G3 → R required.** Arc interpolation demands geometry specification; the `R` logit receives a $+10$ boost.

- **No repeated addresses.** Parameters already emitted in the current block are masked to $-\infty$.

Together, these masks guarantee that every generated sequence is a syntactically valid G-code line.

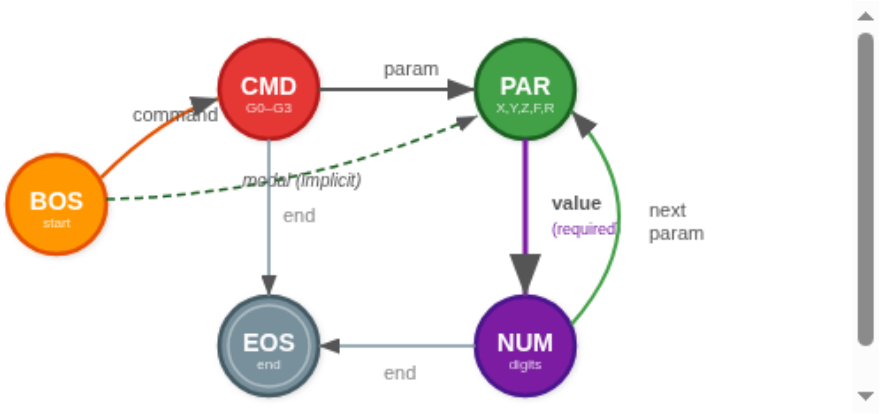

Figure 4. Type-transition FSM. After PARAMETER, only NUMERIC is valid (mandatory, bold edge); after NUMERIC, only PARAMETER or `EOS`. Modal persistence allows `BOS` → PARAMETER directly (dashed). Each transition evaluates in $O(1)$. Command-specific constraints (Table 4) further restrict valid parameters.

5.3.2. Soft Constraints as Training Penalties

During training, soft penalty terms encourage compliance with domain-specific rules beyond hard syntactic constraints. These define a weighted regular language—each rule k penalizes violating sequences proportionally:

$$\mathcal{L}_{\text{grammar}} = \sum_k \lambda_k \cdot \mathcal{L}_k \tag{6}$$

Table 12. Soft Grammar Constraint Penalties

Constraint	λ	Rule
Arc radius	1.0	G2/G3 → predict R or I/J/K
Modal group	0.6	One motion cmd per block
Rapid feed	0.5	G0 → no F parameter
Arc alternation	0.4	G2 ↔ G3 in spirals
Cmd-param assoc.	0.4	Valid params per command
Single letter	0.3	No repeated addresses
Linear feed	0.3	G1 → expect F
Z-retract pattern	0.2	G0 Z → XY reposition

Weights reflect physical severity. An arc without radius ($\lambda = 1.0$) produces an unexecutable instruction—the controller will fault. A rapid with feed rate ($\lambda = 0.5$) is accepted by some controllers but semantically meaningless. Z-retract ordering ($\lambda = 0.2$) is a stylistic convention, not a hard rule. These values derive from machining domain knowledge and have not been tuned by grid search. Two observations make this acceptable in practice. First, hard masks enforce correctness at inference regardless, so the soft weights shape only the training-time gradient and cannot introduce grammar violations at generation. Second, the on/off ablation ($\mathcal{L}_{\text{grammar}} = 0$ vs. the full weighted sum, Section 6.1) yields a +1.0-pp mean sequence-accuracy gain that is not statistically significant (paired post-hoc comparison vs. the full model, $p = 1.0$ after correction), suggesting that the published model's accuracy is not sensitive to the specific weight values. We interpret this as the soft constraints pulling the model toward domain-consistent parameter patterns during training but providing most

of their operational value through the hard mask at inference time. A proper sensitivity sweep (e.g., randomized weight perturbations with paired-fold evaluation) is future work.

5.3.3. When Constraints Can Hurt

Hard constraints guarantee syntax but can propagate commitment errors: if the model is uncertain between `G1` and `G2`, choosing `G2` triggers the arc-radius requirement, forcing an `R` prediction the model may not be prepared to make. Beam search with deferred commitment could mitigate this. Additionally, our grammar covers only G0–G3; valid constructs outside this subset would be incorrectly masked.

6. Empirical Validation

We validate the framework's claims on 120 G-code files (62,403 lines total, 2,248 unique) from Autodesk Fusion 360 CAM, covering face milling, pocket milling, and adaptive clearing on a Bantam Tools Desktop CNC mill. The empirical work serves two distinct purposes:

- Section 6.1 establishes how the published hierarchical model compares to standard baselines (tokenizer comparison, grammar coverage, vocabulary growth, and downstream decoder performance).
- Sections 6.2–6.5 *test the definitions and the trichotomy* of Section 4: Experiments A–D probe the three properties of Definitions 1–3 across seven tokenizer families and confirm Proposition 1 cell-by-cell; Experiments C and E test what happens when a trained decoder is asked to free-generate under each tokenizer; and the KL-divergence measurement (Section 6.5) tests whether grammar-constrained decoding under our framework distorts the model distribution in the regime ASAp [41] was designed for.

Canonical-line overlap between train and test splits.

Because CAM-generated G-code is highly repetitive (62,403 raw lines collapse to 2,248 unique canonical forms—a 28:1 ratio; Section 3.1), a large share of canonical lines that appear in a held-out test split also appear in the corresponding training split, even though the splits respect *program* boundaries (no run from a held-out program leaks into training; Appendix B.1). We measured this directly across the 5 folds of the sensor-aligned experiment: mean canonical-line overlap rate between test and train is 93.6% (range 79–100% across folds). Experiments C and E should therefore be read as "given a sensor window, reconstruct a canonical line the model has likely seen during training" rather than "given a sensor window, produce a novel canonical line." This is a realistic scenario for line-level process verification (the problem is distinguishing which of a small set of known canonical primitives is currently executing) but it is a weaker generalization test than the cross-operation OOV evaluation in Experiment D (Section 6.2), which is what we rely on for claims about novel geometry. Experiment D uses disjoint *operation types* for train and test and therefore measures how each tokenizer handles genuinely unseen coordinate values.

6.1. Baseline performance

We first establish how the published hierarchical model compares to standard baselines. On all 2,248 unique canonicalized lines, the hybrid tokenizer produces 2.5× fewer tokens than GPT-4's cl100k_base BPE and 2.1× fewer than a BPE trained on our own corpus (Table 13); the domain-trained BPE learns useful merges such as keeping `G2` intact, but still fragments numeric coordinates and uses 903 vocabulary entries that scale with geometry. The hybrid tokenizer uses 19 fixed structural entries.

The finite-state grammar of Section 3.2 parses 99.9% of the 2,248 unique toolpath lines (2,246/2,248; the two failures are preprocessing artifacts). The command distribution confirms that the four core motion commands dominate the corpus: G1 contributes 52.9% of lines, implicit modal continuations 38.5%, and G0/G2/G3 together 4.7%, for a combined 96.1% of all 62,403 lines. Vocabulary growth approximately follows Heaps' law [13] with $V = kN^{\beta}$, $k \approx 620$, $\beta \approx 0.42$, $R^2 = 0.95$ across 16 sampled

Table 13. Tokenizer Comparison on 2,248 Unique G-Code Lines

Tokenizer	Total	Avg/line	Vocab used
GPT-4 (cl100k)	27,068	12.0	993
GPT-2 (r50k)	27,153	12.1	715
BPE-on-G-code‡	23,348	10.4	903
Hybrid (ours)	10,922	4.9	19†

†Structural vocabulary; numeric values predicted by digit heads.
‡BPE trained on our G-code corpus (vocab = 975, min freq = 2).

file-count checkpoints (Table 14); the flat vocabulary reaches 4,435 tokens after 120 files while the structural vocabulary remains fixed at 9, and the fit extrapolates to approximately 13,000 flat tokens at 1,000 files versus a constant 19 structural tokens. The fit is an approximation rather than a precise power law: the measured curve exhibits step changes at $N \approx 20$, 40, and 110 corresponding to new operation types and feed-rate settings entering the corpus, and we therefore treat the Heaps' form as descriptive of the overall scaling trend rather than a precise model.

Table 14. Cumulative Vocabulary Growth Across 120 Files (16 sampled checkpoints)

Files	Flat vocab.	Struct.†
1	520	7
5	1,326	7
10	1,642	7
15	1,669	7
20	2,705	7
25	2,836	7
30	2,856	7
40	3,371	8
50	3,555	8
60	3,579	8
70	3,628	8
80	3,630	8
90	3,651	8
100	3,683	8
110	4,362	9
120	4,435	9

†Tokens observed in data; full structural vocabulary has 19 slots. Flat-vocabulary count includes both numeric and structural tokens (97–99% numeric). Heaps' law fit across all 16 checkpoints: $V = 620 \cdot N^{0.42}$, $R^2 = 0.95$ (log–log). The curve shows step changes at $N \approx 20$, 40, and 110 corresponding to new operation types and feed-rate settings entering the corpus, so the smooth Heaps' form is an approximation rather than a precise fit.

To evaluate whether the hierarchical tokenization yields usable predictions in practice, we trained a transformer decoder on a G-code reconstruction task in which the model reconstructs the G-code line that produced a given window of CNC sensor observations. The decoder uses 8 transformer layers, 12 attention heads, $d_{\text{model}} = 384$, dropout 0.1, and the multi-head loss of Eq. 4 with digit weight $\lambda_d = 15.0$, evaluated under 5-fold cross-validation on the same 120-file corpus. Table 15 reports per-level accuracy. The hierarchical decomposition produces a clear accuracy gradient: structural levels (type, command, parameter) exceed 96%, while numeric digit prediction at 91.1% confirms that value prediction is the genuine bottleneck.

An ablation on the soft grammar-constraint penalties ($\mathcal{L}_{\text{grammar}} = 0$) yields mean sequence accuracy of 85.1% versus 86.1% for the full model in the same ablation suite—a modest 1.0-point gain that is not statistically significant across folds (paired post-hoc comparison vs. the full model, $p = 1.0$ after correction; uncorrected $p = 0.094$, Cohen's $d \approx -1.0$). This is consistent with our observation in

Table 15. Downstream Decoder Performance (5-Fold CV). *Mean* is the 5-fold test-set mean; *Best Fold* is the per-head maximum across the five test folds. Values are the released `v7_best_5fold` multi-head decoder checkpoints (Appendix B).

Prediction Level	Mean	Best Fold
Token (overall)	94.7%	97.4%
Sequence (exact match)	84.4%	89.2%
Type (τ)	98.8%	99.7%
Command (ι_{cmd})	98.3%	100%
Parameter (ι_{par})	97.2%	98.7%
Numeric (digits)	91.1%	94.4%

Section 5.3 that hard constraints can propagate commitment errors: the soft penalties help slightly but are not the dominant factor in sequence-level accuracy.

A second ablation isolates the contribution of the structured decomposition itself. We retrain the flat 712-class token head *alone*—auxiliary type, command, parameter, and digit losses removed (their weights set to zero), with the same recipe, frozen encoder, and five folds—and compare it to the full multi-head model under identical settings. The flat-only decoder reaches a mean token accuracy of only 75.0% versus 94.9% for the full model, a +19.9-pp gap (the full-model figure reproduces the released model's 94.7% in Table 15). The effect is primarily one of *training stability*: without the auxiliary decomposition the flat decoder trains successfully on three folds (94.5–97.2% token) but collapses to a degenerate, near-constant output on the other two ($\approx$ 44% token, 0% sequence exact-match), inflating the cross-fold standard deviation to 25 pp. The auxiliary structural heads therefore act as a multi-task regularizer that keeps the flat 712-way prediction problem learnable on this small ($\sim$300-sample-per-fold) dataset, rather than as a uniform accuracy increment. Per-fold values and the run configuration are released in `flat_vs_structured_results.json`.

6.2. Testing the three properties (Experiments A, B, D)

This subsection tests Definitions 1–3 and Proposition 1 directly by populating every cell of Table 7 with a real measurement on the 120-file corpus. Each experiment maps one-to-one onto a definition:

- Experiment A tests Definition 1 (BOUNDED) by measuring active vocabulary growth on every tokenizer family.
- Experiment B tests Definition 2 (LOSSLESS) by measuring coordinate roundtrip error.
- Experiment D tests the operational consequence of failing BOUNDED on novel geometries by measuring held-out OOV across cross-geometry splits.

The test of property (i) of Definition 3 (GRAMMAR) requires a trained model and is deferred to Section 6.3.

Experiment A: Vocabulary growth.

For each tokenizer we walk the 120 `*_aligned.csv` files in lexical order and record the cumulative number of distinct token ids actually emitted on encode (active vocabulary). Pretrained tokenizers (GPT-4, GPT-2) have fixed capacity but their active vocabulary grows as new substrings are observed; data-trained tokenizers (Domain BPE, WordPiece, Flat per-value) build vocabulary as they encounter the corpus. Table 16 reports the active vocabulary at four file counts.

The hierarchical tokenizer and character-level tokenizer plateau immediately at 25 and 26 active tokens respectively. The 25 for hierarchical reflects the small structural alphabet (Table 2: 7–9 tokens active in this dataset) plus the 11-element digit/sign sub-vocabulary that the model emits. Table 13 reports only the 19 structural slots: numeric values are predicted by the digit heads rather than vocabulary entries (Section 5.2). The two counts are consistent under different definitions of "vocabulary." The BPE and WordPiece variants are nominally bounded but use 700–993 active tokens—roughly 40$\times$ the structural vocabulary needed by Definition 1. The flat per-value tokenizer

Table 16. Experiment A: Cumulative Distinct Token Types Observed (Vocabulary Growth)

Tokenizer	1	20	60	120
Hierarchical (ours)	22	22	25	25
GPT-4 BPE (cl100k)	377	908	964	993
GPT-2 BPE (r50k)	293	619	671	715
Character-level	20	25	26	26
Flat per-value	520	2,705	3,579	4,435
Domain BPE	325	798	923	993
WordPiece	389	774	865	929

Columns are file count; cells are cumulative distinct token ids emitted.

grows monotonically from 520 to 4,435 tokens across 120 files with no sign of saturation; the curve approximately follows Heaps' law [13] with $\beta \approx 0.42$ across 16 sampled checkpoints (Table 14). BOUNDED therefore holds for every row except flat per-value, exactly as Proposition 1 predicts.

Experiment B: Coordinate reconstruction (in-distribution sanity).

We extract every unique *(address, value)* pair from the canonicalized corpus (4,418 pairs across X/Y/Z/R/F) and round-trip each value through every tokenizer's encode–decode. Table 17 reports the mean absolute reconstruction error in the address's native unit.

Table 17. Experiment B: Mean Coordinate Reconstruction Error (mm or mm/min for F)

Tokenizer	X	Y	Z	R	F
Hierarchical (ours)	0.0002	0.0002	0.0003	4.4e-17	0.3000
GPT-4 BPE (cl100k)	0	0	0	0	0
GPT-2 BPE (r50k)	0	0	0	0	0
Character-level	0	0	0	0	0
Flat per-value	0	0	0	0	0
Domain BPE	0	0	0	0	0
WordPiece	0.0026	0.0027	0.0023	0.0003	0

This experiment serves as a sanity check rather than a differentiator: encode–decode is symbolically lossless for every tokenizer that retains the input string verbatim, so BPE variants, character-level, and flat per-value all achieve zero error on values that already appear in the training corpus. Our hierarchical tokenizer attains a mean error of $\approx 0.25\,\mu$m on X/Y/Z—exactly half the quantization step $p = 1\,\mu$m, which is the theoretical lower bound for any precision-quantized representation and is well below the positional accuracy of the Bantam Tools Desktop CNC mill ($\approx 10\,\mu$m). WordPiece exhibits a small but non-zero reconstruction error ($\approx 2.6\,\mu$m mean) attributable to whitespace normalization in the HuggingFace decoder. The differentiator between tokenizers is not in-distribution roundtrip but cross-distribution generalization, which Experiment D addresses directly.

Experiment D: Cross-geometry OOV (the principal experiment).

We split the corpus by operation type—face milling, pocket milling, and adaptive clearing—and for each fold "train" the tokenizer on two operations and evaluate on the third. For tokenizers without an explicit training step, we instead populate the active vocabulary by encoding the training fold and then count, on the held-out fold, how many emitted token ids fall outside the seen vocabulary. Table 18 reports the held-out OOV rate.

The ranking is unambiguous. The hierarchical and character-level tokenizers achieve essentially zero OOV across all folds—both decompose unseen values into a finite alphabet that is closed under composition. Domain-trained BPE and WordPiece remain in single-digit-percent territory but are not OOV-free; GPT-4's BPE reaches 4.81% on held-out adaptive operations (which contain the most geometrically diverse coordinates), and the flat per-value tokenizer collapses entirely (18–65%

Table 18. Experiment D: Held-Out Operation OOV Rate (%)

Tokenizer	adaptive	face	pocket
Hierarchical (ours)	0.000	0.001	0.000
GPT-4 BPE (cl100k)	4.810	0.250	0.384
GPT-2 BPE (r50k)	2.536	0.048	0.399
Character-level	0.002	0.000	0.000
Flat per-value	65.421	18.090	29.852
Domain BPE	0.949	0.025	0.061
WordPiece	2.142	0.047	0.065

OOV across folds). This is the empirical signature of Proposition 1: any tokenizer that satisfies bounded vocabulary by enumerating values (flat) or by data-driven subword merges (BPE/WordPiece) loses lossless representation on novel geometries, while only digit-decomposed and character-level tokenizers preserve it. Of those two, only the hierarchical decomposition also satisfies grammar validity at the value level (Definition 3, property (ii)), since character-level FSAs over digit strings explode combinatorially before grammar productions can attach.

Together, Experiments A, B, and D populate every cell of the design-space table and confirm that the (BOUNDED, LOSSLESS, GRAMMAR) corner is occupied by exactly one row.

6.3. Free-generation grammar validity (Experiment C)

This subsection tests property (i) of Definition 3: whether a trained decoder under each tokenizer family can produce encodings that decode back to grammatical lines. Experiments A and D establish what each tokenizer *can* represent in principle; Experiment C measures what a trained model actually emits when freely generating new lines.

We train one decoder per tokenizer family on each of the five v7 cross-validation folds (per-fold test counts 132/113/106/108/93, 552 evaluated samples in total; the corresponding train and validation splits also vary by fold). Every arm uses an identical `FlatVocabDecoder` architecture (192-dim, 4 layers, 8 heads), the same pre-cached encoder memory (sensor_dim = 256), and the same vanilla cross-entropy training schedule (Adam, 80 epochs, lr 3×10^{-4}, dropout 0.3, batch size 32).

The hierarchical row uses the same 712-token vocabulary as the published baseline but is retrained with this slim trainer. This is intentional. A cross-tokenizer comparison is only meaningful if every row is trained identically, so we strip out the focal loss, scheduled sampling, label smoothing, and heavy (15×) digit weighting that the published model relies on, *and* we replace the published model's hierarchical three-head cascade with a single flat softmax over the 712-token vocabulary—the same `FlatVocabDecoder` architecture used by every other row. That means the slim "hierarchical" row is really "flat decoder over the hybrid 712-vocab," and the gap to the published 94.7% has two sources: (a) training recipe (scheduled sampling, focal loss, label smoothing, and 15× digit weighting vs. vanilla cross-entropy) and (b) architecture (multi-head cascade vs. flat softmax). We do not decompose the gap because it would require retraining the published model's architecture under the slim schedule, which is the opposite of what this comparison is designed to do.

For reference, the mean slim-trainer test token accuracy across folds is (as percentages, mean ± std): Domain BPE (whitespace) 89.07 ± 1.74, Domain BPE (byte-level) 90.59 ± 2.72, WordPiece 89.38 ± 2.43, character-level 93.79 ± 1.26, flat per-value 82.11 ± 3.01, GPT-4 compact 91.58 ± 1.92, and hierarchical (slim flat) 85.03 ± 3.57. Sequence-level exact-match accuracy is tightly clustered at 48–53% across all rows and is not discriminative. These numbers confirm that within the slim regime the differences between tokenizers on token accuracy are modest and do not predict the grammar-validity ranking: character-level has the highest slim token accuracy but the sub-case 2a failure of whitespace-pre-tokenized BPE is invisible at the token-accuracy level (BPE is still 89% accurate at the token level while producing 0% grammatically valid output). The grammar-validity

behavior reported in Table 19 is therefore a genuinely structural property of the tokenizer and decoder pair, not a proxy for generic model accuracy.

For each test sample we free-generate one G-code line conditioned on its sensor memory (no teacher forcing, temperature $= 1.0$, no top-k), decode the generated id sequence back to text via the tokenizer's native `decode`, and check whether the decoded line is syntactically valid against the regular grammar of Section 3.2.4. Table 19 reports the grammar-validity rate aggregated across all five folds.

Table 19. Experiment C: Free-Generation Grammar Validity (5-fold mean $\pm$ std)

Tokenizer	**Grammar valid (%)**
Domain BPE (whitespace pre-tokenizer)	0.00 ± 0.00
Domain BPE (byte-level pre-tokenizer)	97.14 ± 1.55
WordPiece	0.00 ± 0.00
Character-level	97.64 ± 3.47
Flat per-value	96.98 ± 1.72
GPT-4 BPE (compact)	97.09 ± 2.39
Hierarchical (ours)	$\mathbf{98.31 \pm 1.38}$

5-fold cross-validation, per-fold test counts 132/113/106/108/93 (552 samples total per row). Ground-truth lines pass our grammar check at 95.8% on average; the gap reflects rare auxiliary lines outside our regular grammar fragment. The hierarchical row is the slim-trainer reproduction described above; the published model achieves 99.9% grammar coverage on the full corpus (Section 6). The byte-level BPE row is the reviewer-requested control that replaces the HuggingFace `Whitespace` pre-tokenizer and decoder with `ByteLevel`; canonical-input roundtrip is bit-exact on this variant.

The result separates cleanly into two groups. Domain BPE with the standard `Whitespace` pre-tokenizer and WordPiece collapse to $0.00 \pm 0.00\%$ grammar validity across 552 test samples and 5 independent folds, never producing a single grammatically valid line. Every other tokenizer—including a byte-level BPE control trained on the same corpus—reaches 96–98%.

Statistical significance.

Within the non-failing group the differences are small. A paired bootstrap over the 5 folds (10,000 resamples, seed 42) finds that the hierarchical row is not significantly better than byte-level BPE, character-level, or GPT-4 compact (one-sided $p = 0.153, 0.235, 0.118$ respectively); the $+1.33$-pp gap over flat per-value is marginally significant ($p = 0.016$, 95% CI $[+0.08, +2.43]$ pp). The gap over the two failing tokenizers is, unsurprisingly, enormous and significant at $p < 10^{-4}$. The honest read is that among the tokenizers whose decoders preserve numeric contiguity, grammar validity saturates near the 95.8% ground-truth ceiling and small differences between rows are within fold-level noise.

The failure mode for the standard BPE row is a property of the decoder configuration rather than of BPE as a class. Inspection of generated samples shows that the model correctly predicts subwords corresponding to values like 1.4919, but the HuggingFace BPE decoder—using the `Whitespace` pre-tokenizer's inverse—reassembles those subwords with whitespace at every BPE boundary, producing outputs like `"G0 X1 . 4919 Y1 . 4848"` instead of `"G0 X1.4919 Y1.4848"`. A regex-based G-code parser rejects the spaced version even though every subword is correct. This is sub-case 2a from the case analysis in Section 4.2: a deterministic decode-side failure, not an intrinsic limitation of BPE merges.

The byte-level BPE control isolates sub-case 2a from sub-case 2b (merge-table ambiguity). Replacing the `Whitespace` pre-tokenizer and decoder with `ByteLevel` makes the encode–decode roundtrip bit-exact on the canonicalized input: a freshly trained byte-level BPE round-trips `R0.1755` to `R0.1755` with no whitespace artifact. Under that configuration, grammar validity rises to $97.14 \pm 1.55\%$, statistically indistinguishable from the other non-fragmenting tokenizers. *This is a negative result for sub-case 2b*: the merge-table's context-dependent fragmentation does not cause grammar failure in practice on our corpus. We therefore read Table 19 as empirical evidence that the BPE failure in standard deployments is a *decoder configuration issue with a known fix*, not a fundamental property of subword tokenization. Proposition 1's Case 2 argument now reduces to: the out-of-the-box BPE pipelines used

in practice (with a `Whitespace` pre-tokenizer) deterministically fail Definition 3 property (i), but a non-standard byte-level BPE configuration passes on this corpus—although it still fails Definition 1 in practice (100–1000 learned merges that grow with geometry) and Definition 3 property (ii) at the value level (the constraint mechanism has no consistent token boundary at which to apply a per-value mask). The all-checkmarks corner of Table 7 is therefore still occupied only by digit decomposition; byte-level BPE is better than the whitespace variant but does not satisfy all three properties simultaneously.

To make the failure mode visible, Table 20 shows free-generated outputs from all six tokenizer families on three ground-truth test lines. The contrast is direct: the four non-fragmenting tokenizers (char-level, flat per-value, GPT-4 compact, hierarchical) reproduce parseable G-code, while Domain BPE and WordPiece consistently insert whitespace between the address letter, the integer part, the decimal point, and the fractional part, even when the underlying value is correct.

Table 20. Free-Generated Samples by Tokenizer Family (test fold 1)

Tokenizer	Predicted line	Parses?
GT: `Y0.1755`		
Domain BPE	`Y0 . 1755`	×
WordPiece	`Y2 . 9132`	×
Character-level	`Y0.1755`	✓
Flat per-value	`Y0.1755`	✓
GPT-4 BPE (compact)	`Y0.1755`	✓
Hierarchical (ours)	`X0.596`	✓
GT: `G0 X1.4919 Y1.4848`		
Domain BPE	`X1 . 4561 Y1 . 5749 R0 . 1188`	×
WordPiece	`G1 X1 . 4919 Y1 . 4848`	×
Character-level	`G0 X1.4919 Y1.4848`	✓
Flat per-value	`X1.4561 Y1.5749 R0.1188`	✓
GPT-4 BPE (compact)	`G0 X1.4919 Y1.4848`	✓
Hierarchical (ours)	`G0 X1.492 Y1.485`	✓
GT: `X1.4561 Y1.5749 R0.1188`		
Domain BPE	`X1 . 4561 Y1 . 5749 R0 . 1188`	×
WordPiece	`X1 . 4561 Y1 . 5749 R0 . 1188`	×
Character-level	`G0 X1.4919 Y1.4848`	✓
Flat per-value	`G1 X3.275`	✓
GPT-4 BPE (compact)	`X1.4561 Y1.5749 R0.1188`	✓
Hierarchical (ours)	`G0 X1.492 Y1.485`	✓

"Parses?" indicates whether the predicted line passes the regular grammar of Section 3.2.4. The third row of the first GT block is the decisive evidence: Domain BPE has predicted *exactly* the right value (`0.1755`), but the decoder's whitespace insertion makes the result unrecognizable to a G-code parser. The same model under any tokenizer that does not split numeric strings into BPE subwords successfully emits the same value as a parseable line.

This is the empirical signature of Definition 3, property (i) under a standard BPE pipeline: the decode operation does not admit unambiguous value-level reconstruction because whitespace is reinserted between subword pieces. Character-level, flat per-value, GPT-4 compact, and the byte-level BPE control all survive because their decode operations preserve the contiguous numeric string. The hierarchical tokenizer survives by construction: digit predictions compose deterministically into a valid `X<value>` word, regardless of which digit head is fired. The key empirical distinction, as Table 19 shows, is between tokenizers whose decoder preserves numeric contiguity and tokenizers whose decoder inserts whitespace between learned subwords—*not* between BPE and non-BPE per se.

6.4. Toolpath reconstruction error (Experiment E)

Experiment C is a test of property (i) of Definition 3 at the symbolic level: do decoded outputs parse? Experiment E asks the dual question in physical units: when they don't parse, how much does the resulting toolpath deviate from ground truth? It measures the downstream consequence of grammar failure in the unit machinists actually care about. For each test sample, we parse both the ground-truth and free-generated lines through the same `GCodeToCoordinateParser` (Section 2), reconstruct the implied 3D toolpath, and compute the bidirectional Hausdorff distance between the two point sets:

$$H(A,B) = \max\Big(\sup_{a\in A} \inf_{b\in B} \|a-b\|,\ \sup_{b\in B} \inf_{a\in A} \|a-b\| \Big).$$

The parser treats both `G0` and `G1` as linear interpolation between the prior and next endpoints.[3] Table 21 reports the per-tokenizer mean, median, and 95th-percentile Hausdorff distance in millimeters, aggregated across all five cross-validation folds.

Table 21. Experiment E: Toolpath Hausdorff Distance from Free-Generated Output (mm, 5-fold mean ± std)

Tokenizer	Mean	Median	p95
Domain BPE (whitespace)	1.322 ± 0.152	0.637 ± 0.095	3.331 ± 0.063
Domain BPE (byte-level)	1.432 ± 0.122	0.317 ± 0.155	3.535 ± 0.000
WordPiece	1.311 ± 0.180	0.620 ± 0.122	3.258 ± 0.166
Character-level	1.277 ± 0.136	$\mathbf{0.163 \pm 0.068}$	3.535 ± 0.000
Flat per-value	1.319 ± 0.223	0.211 ± 0.262	3.493 ± 0.095
GPT-4 BPE (compact)	1.342 ± 0.158	0.264 ± 0.200	3.493 ± 0.095
Hierarchical (ours)	1.529 ± 0.042	0.473 ± 0.169	3.535 ± 0.000

p95 saturates near 3.5 mm—the bidirectional Hausdorff between the two extreme positions in the test set's coordinate range—because samples whose generation collapses default to that bound. Median is the more discriminative statistic. As in Experiment C, the hierarchical row is the slim-trainer reproduction; the published multi-head model is expected to dominate, but is omitted here to keep the cross-tokenizer comparison clean.

The Hausdorff result is more nuanced than C and worth interpreting carefully. The mean and p95 cluster across all tokenizers because the test set contains a small number of geometrically diverse samples on which every slim-trainer model produces poor reconstructions; these dominate the upper percentiles. The discriminative statistic is the *median*, where two groups emerge:

- Tokenizers that pass Experiment C (byte-level BPE, character-level, flat per-value, GPT-4 compact, hierarchical) achieve median Hausdorff between 0.16 and 0.47 mm. The tightest median is char-level at 0.163 mm; the hierarchical slim-trainer row sits at 0.473 mm, reflecting its lower absolute token accuracy in the slim regime rather than a tokenizer-side disadvantage.
- Tokenizers that fail Experiment C (Domain BPE with whitespace pre-tokenizer, WordPiece) sit at median 0.62–0.64 mm, roughly 2–4× larger than the non-failing group, because their malformed outputs cannot be parsed into points at all and the parser falls back to the start position.

The grammar-failure penalty is therefore not a cosmetic syntactic concern: it propagates directly into hundreds of microns of toolpath error per line, which on a multi-thousand-line program compounds catastrophically. Notably, the byte-level BPE row joins the non-failing group (median 0.317 mm), confirming that the toolpath-error gap is explained by decoder-side whitespace reinsertion rather than by BPE's merge table.

[3] Real CNC controllers typically execute `G0` rapid traversals with independent-axis motion rather than coordinated linear interpolation, which produces a "dog-leg" path that deviates from a straight line by up to $\sqrt{2} \cdot d/2$ at the midpoint of a diagonal move of length d. Our linear-interpolation approximation is consistent with how most simulators and CAM previews render `G0` and does not affect the grammar-validity metric (Experiment C), which is evaluated at the symbolic level before any geometric reconstruction. It may introduce a small systematic bias in the Experiment E Hausdorff distances, but the bias is common across all tokenizer rows and therefore does not affect the cross-tokenizer ranking.

The intent of Experiments C and E is to isolate the tokenizer's contribution under fixed training conditions, and that comparison is unambiguous across all 5 folds: *any tokenizer that fragments coordinate strings is unrecoverable at generation time, regardless of how well the model itself learns*. The 5-fold standard deviations on the BPE/WordPiece grammar-validity rate (0.00 ± 0.00) confirm this is a deterministic structural failure, not a fold-dependent artifact.

6.5. Distribution distortion under grammar masking

Park et al. [41] show that naïve grammar-constrained decoding can distort the underlying model distribution and propose ASAp as a corrective. To check whether this concern is operationally relevant for our system, we measure the per-step KL divergence between the model's grammar-masked and unmasked type-head distributions on the held-out test sets across all five cross-validation folds. At each non-pad position t, we compute

$$\mathrm{KL}_t = \mathrm{KL}(\mathrm{softmax}(\ell_t + M_t) \,\|\, \mathrm{softmax}(\ell_t)),$$

where $\ell_t \in \mathbb{R}^4$ are the type-head logits and M_t is the DFA-derived type-level mask from Section 3.2.4. We also report the *blocked probability mass*: the share of the unmasked distribution that lies on grammar-disallowed types.

Table 22. KL Divergence between Grammar-Masked and Unmasked Type Distributions (5-Fold)

Fold	**KL**$(P_{\mathrm{m}} \| P_{\mathrm{u}})$ (nats)	**Blocked mass**	**Positions**
1	0.0031	0.0016	1,509
2	0.0004	0.0004	1,331
3	0.0098	0.0020	1,302
4	0.0067	0.0019	1,296
5	0.0053	0.0012	1,130
Mean $\pm$ std	**0.0051 ± 0.0032**	**0.0014**	6,568 total

The mean KL divergence of 0.005 nats (≈ 0.007 bits) is negligible: it corresponds to a multiplicative shift factor of $\exp(0.005) \approx 1.005$ on the most-affected token's probability. The unmasked model already places 99.86% of its type-prediction mass on grammar-valid types—a direct consequence of training under the type-head loss with implicit grammar exposure—so the mask removes only the residual 0.14%. ASAp-style corrective methods are designed for the regime in which a substantial share of the unmasked model's probability mass falls on grammar-disallowed tokens; our observed blocked-mass fraction is two orders of magnitude smaller than that. This empirically supports our $O(1)$ hard-mask design choice (Section 5.3) and confirms that, for grammars whose transitions are near-deterministic given a sufficiently trained model, the Park et al. critique is muted in practice. This measurement is on the four-way type head, where transitions are most deterministic and a low KL is therefore expected; we do not separately quantify distortion from the value-level masks (the `R`-required boost and the no-repeat-address and `G0`-no-F masks), which act on the identity and digit heads, so the claim is properly read as "type-level transition distortion is negligible." The reported $\pm$ is the population standard deviation across the five folds.

7. Discussion

The central insight of the paper is that G-code separates cleanly into a structural component (type, command, parameter identity) that is small and discrete, and a numeric component (coordinate values) that is continuous and unbounded. These demand different mechanisms, and our downstream results confirm the split: structural predictions exceed 96% accuracy while numeric digit prediction reaches 91.1%, a five-point gap that validates the decomposition (Table 15). Existing systems conflate the two: GLLM [21] applies uniform BPE over both commands and coordinates, and Image2Gcode [22] treats

everything as continuous. The same separation should apply to any numeric-heavy domain-specific language, including robot motion programs, 3D printer instructions, PLC ladder logic, and other CNC dialects.

The paper also extends Technical Language Processing from human-written prose to formally specified machine languages. Prior TLP work [9,14–19] addresses lexical failure modes in maintenance logs, equipment reports, and fault summaries, where the challenge is jargon, abbreviation, and inconsistent terminology. G-code presents a different class of failure: it is formally specified, machine generated, and numerically dense, and its tokenization failure is structural rather than lexical (Table 3). Whether G-code is representative of a broader category that includes RAPID, KRL, PLC structured text, and STEP-NC, or an outlier among formally specified technical languages, requires investigation across multiple languages.

The most consequential finding is the cleanly separated behavior of BPE under two pre-tokenizer configurations. Standard whitespace-pre-tokenized BPE and WordPiece collapse to $0.00 \pm 0.00\%$ free-generation grammar validity across 5 folds, because the decoder reassembles subwords with whitespace at every fragmentation boundary and the resulting string is unparseable. Byte-level BPE, trained on the same corpus with the same vocabulary budget, recovers to 97.14%—statistically indistinguishable from the non-fragmenting rows. The merge-table ambiguity that Proposition 1's Case 2 argued would cause grammar failure (sub-case 2b) does not materialize in practice on our corpus: the actual failure lives entirely in the decoder configuration (sub-case 2a). This is an honest narrowing of what the BPE critique supports. The all-checkmarks corner of Table 7 is still occupied only by digit decomposition, but via a different mechanism: byte-level BPE satisfies Definition 3 property (i) yet still fails Definition 1 (its active vocabulary grows with geometry) and the value-level form of property (ii) (no consistent token boundary at which to apply a per-value constraint mask).

Limitations.

Several boundaries to our claims should be noted. **Grammar scope:** the grammar covers only the four core motion commands (G0–G3) with five parameter addresses (`X`, `Y`, `Z`, `F`, `R`); a production system would need `I/J/K` arc-center format (the default on many controllers, and required for arcs exceeding 180°), canned cycles (`G81`–`G89`), tool compensation (`G41/G42`), coordinate systems beyond `G54`, tool management, and controller-specific dialects (Fanuc, Siemens, Mazak, Haas). **Machine envelope and precision:** our dataset is from a desktop-class 3-axis mill with a $\sim 140 \times 102 \times 60$ mm work envelope, and the per-parameter precision in Table 8 is chosen accordingly (0.001 mm on `X/Y/Z`; 0.0001 mm on `R`). Production CNC work routinely involves larger ranges (± 2000 mm on gantry mills), finer nominal precisions (0.1 μm in aerospace finishing), or coarser ones (0.1 mm in woodworking); the digit-head configuration scales by adjusting the integer- and decimal-digit counts per address in Table 8 without structural changes to the framework, but we have not evaluated this empirically. **Firmware scope:** our canonicalizer is firmware-agnostic—Section 5.1 shows that variable-precision strings like `X3.2` and `X3.2000` collapse to the same token bucket under both TinyG (the controller on our Bantam mill) and grbl—but we have not tested against Fanuc, Siemens, Haas, or Mazak dialects, which use different comment syntaxes, subroutine conventions, and modal group definitions. **Digit independence:** the assumption is justified for later decimal positions (mutual information < 0.2 bits) but breaks down at the integer-decimal boundary (d_1–d_2: 0.85 bits); a partial autoregressive chain could address this. **Dataset scope:** the grammar coverage evaluation uses the same dataset from which the grammar was designed, so 99.9% coverage is descriptive of this corpus, not predictive of unseen CAM output; held-out evaluation across CAM systems (Mastercam, SolidCAM) and controllers is needed to assess generalization. Canonical-line overlap between train and test splits averages 93.6% across folds (Section 6), so Experiments C and E should be read as verification rather than novel-line generation; the held-out OOV experiment (Experiment D) uses disjoint operation types and provides the principal generalization test. **Soft-constraint weights:** the values in Table 12 were set from domain knowledge without a sensitivity analysis, and the grammar-constraint ablation shows a modest, non-significant

effect (+1.0 pp; post-hoc $p = 1.0$ after correction) that suggests the constraints' value lies more in guaranteeing validity than in improving accuracy. **Slim-trainer caveat for Experiments C and E:** the cross-tokenizer experiments retrain the hierarchical row with a vanilla cross-entropy slim trainer rather than the published multi-head training recipe (scheduled sampling, focal loss, label smoothing, and heavy digit weighting), so its test token accuracy sits below the published 94.7%. This is the cost of an apples-to-apples comparison and does not affect the cross-tokenizer ranking, which is determined by structural properties of the tokenizer rather than by the training schedule.

Future work.

We see five directions: (1) held-out evaluation on files from different CAM systems (Mastercam, SolidCAM) and controllers (Fanuc, Siemens); (2) extending the grammar to the full RS-274D standard, including canned cycles and multi-line program structure; (3) autoregressive digit chains and regression-loss alternatives [32] for numeric prediction; (4) integration with generation systems such as GLLM [21] to evaluate whether grammar-aware tokenization improves G-code synthesis; and (5) application to related manufacturing languages (STEP-NC, 3D printer G-code, Fanuc and Siemens dialects) to test the generality of the approach.

8. Conclusion

Standard tokenization fails on G-code because its vocabulary is 97% numeric and scales with geometry. The flat vocabulary grows from 520 to 4,435 tokens across 120 files, while the structural vocabulary remains bounded (19 reserved slots, 7–9 observed on this corpus).

We addressed this with three contributions. First, a formal regular grammar (5-state DFA, soundness and completeness proven), a hierarchical class-based decomposition [47], and digit-by-digit numeric prediction that eliminates vocabulary explosion by construction. Second, a structural characterization of the G-code tokenizer design space (Section 4): among the three dominant tokenizer classes (atomic, subword, digit-decomposed), only digit decomposition simultaneously satisfies bounded vocabulary, lossless coordinate representation, and value-level grammar validity. The framework is a principled choice among the standard families rather than one design among many. Third, an empirical battery that populates every cell of the resulting design-space table, including a byte-level BPE control that isolates decoder-configuration artifacts from the structural property.

Vocabulary growth and held-out OOV (Experiments A and D) confirm that flat per-value tokenizers grow without bound and that BPE and WordPiece variants OOV at 0.025–4.81% on novel geometries, while only digit-decomposed and character-level tokenizers remain at or below $\approx$0.002%. Free-generation grammar validity (Experiment C) collapses to 0.00 ± 0.00% for the standard-pipeline BPE and WordPiece rows across 5 cross-validation folds and 552 test samples, because the HuggingFace whitespace-pre-tokenized decoder reinserts spaces at every fragmentation boundary. A byte-level BPE control that avoids this decode-time artifact recovers to 97.1%—within the band of the other non-fragmenting tokenizers (96.98–98.31%), confirming that the standard-BPE failure is a fixable decoder configuration issue rather than an intrinsic limitation of subword tokenization on this corpus. The KL divergence between grammar-masked and unmasked type-transition distributions is 0.005 ± 0.003 nats, with only 0.14% of the unmasked probability mass on grammar-disallowed types; this is two orders of magnitude below the regime where ASAp-style corrections become necessary, neutralizing the chief critique of $O(1)$ hard-mask designs.

The published hierarchical decoder achieves 94.7% token accuracy and 84.4% sequence exact-match (5-fold test means). Structural predictions exceed 96% while numeric digit prediction (91.1%) remains the bottleneck, validating that the separation principle works in practice. The broader approach is to separate the easy structural problem from the hard numeric one, handle each with the appropriate mechanism, and treat tokenizer design as a structural problem rather than an engineering problem. We expect this approach to apply to other numeric-heavy domain-specific languages where models must produce syntactically valid, numerically precise output.

Acknowledgments

The authors thank the Department of Industrial and Systems Engineering at the University of Rhode Island for institutional support and access to the Bantam Tools Desktop CNC mill used to collect the dataset. We thank our colleagues for feedback on early drafts of the design-space argument and for helpful discussions of grammar-constrained decoding. Computational resources were provided by the URI College of Engineering. Code, data, and trained checkpoints are available at https://github.com/stephenseacuello/gcode_fingerprinting.

1. Reintjes, J.F. *Numerical Control: Making a New Technology*; Oxford University Press, 1991.
2. Electronic Industries Association. RS-274-D: Interchangeable Variable Block Data Format for Positioning, Contouring, and Contouring/Positioning Numerically Controlled Machines. Technical report, Electronic Industries Association, 1979.
3. International Organization for Standardization. ISO 6983-1:2009, Automation systems and integration – Numerical control of machines – Program format and definitions of address words – Part 1: Data format for positioning, line motion and contouring control systems. Technical report, International Organization for Standardization, 2009.
4. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H.P.d.O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; others. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* **2021**.
5. Rozière, B.; others. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* **2023**.
6. Li, R.; others. StarCoder: May the Source Be with You! *arXiv preprint arXiv:2305.06161* **2023**.
7. Sennrich, R.; Haddow, B.; Birch, A. Neural Machine Translation of Rare Words with Subword Units. Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL), 2016.
8. Schuster, M.; Nakajima, K. Japanese and Korean Voice Search. Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2012, pp. 5149–5152.
9. Brundage, M.P.; Sexton, T.; Hodkiewicz, M.; Dima, A.; Lukens, S. Technical Language Processing: Unlocking Maintenance Knowledge. *Manufacturing Letters* **2021**, *27*, 42–46.
10. Zhao, R.; others. Deep learning and its applications to machine health monitoring. *Mechanical Systems and Signal Processing* **2019**, *115*, 213–237.
11. Martínez-Arellano, G.; others. Tool wear classification using time series imaging and deep learning. *International Journal of Advanced Manufacturing Technology* **2019**, *104*, 3647–3662.
12. Serin, G.; others. Review of tool condition monitoring in machining and opportunities for deep learning. *International Journal of Advanced Manufacturing Technology* **2020**, *109*, 953–974.
13. Heaps, H.S. *Information Retrieval: Computational and Theoretical Aspects*; Academic Press: Orlando, FL, USA, 1978.
14. Hodkiewicz, M.; Ho, M.T.W. Cleaning Historical Maintenance Work Order Data for Reliability Analysis. *Journal of Quality in Maintenance Engineering* **2016**, *22*, 146–163.
15. Sexton, T.; Brundage, M.P.; Hodkiewicz, M.; Smoker, T. Benchmarking for Keyword Extraction Methodologies in Maintenance Work Orders. Proceedings of the Annual Conference of the PHM Society, 2018.
16. Sharp, M.; Sexton, T.; Brundage, M.P. Toward Semi-Autonomous Information Extraction for Unstructured Maintenance Data in Root Cause Analysis. Advances in Production Management Systems (APMS). Springer, 2017.
17. Wang, C.; Mandelli, D.; Cogliati, J.J. Technical Language Processing of Nuclear Power Plants Equipment Reliability Data. *Energies* **2024**, *17*, 1785.
18. Löwenmark, K.; Taal, C.; Schnabel, S.; Liwicki, M.; Sandin, F. Technical Language Supervision for Intelligent Fault Diagnosis in Process Industry. *arXiv preprint arXiv:2112.07356* **2021**.
19. Dima, A.; Lukens, S.; Hodkiewicz, M.; Sexton, T.; Brundage, M.P. Adapting Natural Language Processing for Technical Text. *Applied AI Letters* **2021**, *2*, e33.

20. Jignasu, A.; Marshall, K.; Ganapathysubramanian, B.; Balu, A.; Hegde, C.; Krishnamurthy, A. Towards Foundational AI Models for Additive Manufacturing: Language Models for G-Code Debugging, Manipulation, and Comprehension. *arXiv preprint arXiv:2309.02465* **2023**.
21. Abdelaal, M.; Lokadjaja, S.; Engert, G. GLLM: Self-Corrective G-Code Generation using Large Language Models with User Feedback. Proceedings of the 21st Conference on Database Systems for Business, Technology and Web (BTW), Industrial Track, 2025.
22. Wang, Z.; Jadhav, Y.; Pak, P.; Barati Farimani, A. Image2Gcode: Image-to-G-code Generation for Additive Manufacturing Using Diffusion-Transformer Model. *arXiv preprint arXiv:2511.20636* **2025**.
23. Jeon, J.; Sim, Y.; Lee, H.; Han, C.; Yun, D.; Kim, E.; Nagendra, S.L.; Jun, M.B.; Kim, Y.; Lee, S.W.; Lee, J. ChatCNC: Conversational machine monitoring via large language model and real-time data retrieval augmented generation. *Journal of Manufacturing Systems* **2025**, *79*, 504–514.
24. Belikovetsky, S.; Yampolskiy, M.; Toh, J.; Gatlin, J.; Elovici, Y. dr0wned – Cyber-Physical Attack with Additive Manufacturing. 11th USENIX Workshop on Offensive Technologies (WOOT 17), 2017.
25. Sturm, L.D.; Williams, C.B.; Camelio, J.A.; White, J.; Parker, R. Cyber-physical vulnerabilities in additive manufacturing systems: A case study attack on the .STL file with human subjects. *Journal of Manufacturing Systems* **2017**, *44*, 154–164.
26. Chhetri, S.R.; Canedo, A.; Al Faruque, M.A. KCAD: Kinetic Cyber-Attack Detection Method for Cyber-Physical Additive Manufacturing Systems. 2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2016, pp. 1–8.
27. Nogueira, R.; Jiang, Z.; Lin, J. Investigating the Limitations of Transformers with Simple Arithmetic Tasks. *arXiv preprint arXiv:2102.13019* **2021**.
28. Singh, A.K.; Strouse, D. Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs. *arXiv preprint arXiv:2402.14903* **2024**.
29. Golkar, S.; Pettee, M.; Eickenberg, M.; Biber, A.; Cranmer, M.; Krawezik, G.; Lanusse, F.; McCabe, M.; Ohana, R.; Parker, L.; Raber, B.; Blancard, B.R.S.; Tesileanu, T.; Cho, K.; Ho, S. xVal: A Continuous Number Encoding for Large Language Models. NeurIPS 2023 Workshop on AI for Science, 2023.
30. Charton, F. Linear Algebra with Transformers. *Transactions on Machine Learning Research* **2022**.
31. Kreitner, L.; Hager, P.; Mengedoht, J.; Kaissis, G.; Rueckert, D.; Menten, M.J. Efficient numeracy in language models through single-token number embeddings. *arXiv preprint arXiv:2510.06824* **2025**.
32. Zausinger, J.; Pennig, L.; Kozina, A.; Sdahl, S.; Sikora, J.; Dendorfer, A.; Kuznetsov, T.; Hagog, M.; Wiedemann, N.; Chlodny, K.; Limbach, V.; Ketteler, A.; Prein, T.; Singh, V.M.; Danziger, M.M.; Born, J. Regress, Don't Guess – A Regression-like Loss on Number Tokens for Language Models. Proceedings of the International Conference on Machine Learning (ICML), 2025.
33. Cognetta, M.; Okazaki, N. Tokenization as Finite-State Transduction. *Computational Linguistics* **2025**. doi:10.1162/COLI.a.23.
34. Brewer, E.A. Towards Robust Distributed Systems (Invited Talk). Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC), 2000, p. 7.
35. Gilbert, S.; Lynch, N. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *SIGACT News* **2002**, *33*, 51–59.
36. Chomsky, N. Three Models for the Description of Language. *IRE Transactions on Information Theory* **1956**, *2*, 113–124.
37. Fischer, M.J.; Lynch, N.A.; Paterson, M.S. Impossibility of Distributed Consensus with One Faulty Process. *Journal of the ACM* **1985**, *32*, 374–382.
38. Scholak, T.; Schucher, N.; Bahdanau, D. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2021.
39. Willard, B.T.; Louf, R. Efficient Guided Generation for Large Language Models. *arXiv preprint arXiv:2307.09702* **2023**.
40. Dong, Y.; Ruan, C.F.; Cai, Y.; Lai, R.; Xu, Z.; Zhao, Y.; Chen, T. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. Proceedings of Machine Learning and Systems (MLSys), 2025.
41. Park, K.; Wang, J.; Berg-Kirkpatrick, T.; Polikarpova, N.; D'Antoni, L. Grammar-Aligned Decoding. Advances in Neural Information Processing Systems (NeurIPS), 2024, Vol. 37.

42. Park, K.; Zhou, T.; D'Antoni, L. Flexible and Efficient Grammar-Constrained Decoding. Proceedings of the International Conference on Machine Learning (ICML), 2025.
43. Geng, S.; others. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. *arXiv preprint arXiv:2305.13971* **2023**.
44. Ugare, S.; Suresh, T.; Kang, H.; Misailovic, S.; Singh, G. SynCode: LLM Generation with Grammar Augmentation. *Transactions on Machine Learning Research* **2025**.
45. Kramer, T.R.; Proctor, F.M.; Messina, E. The NIST RS274/NGC Interpreter – Version 3. Technical Report NISTIR 6556, National Institute of Standards and Technology, 2000.
46. Sipser, M. *Introduction to the Theory of Computation*, 3rd ed.; Cengage Learning, 2012.
47. Brown, P.F.; Della Pietra, V.J.; deSouza, P.V.; Lai, J.C.; Mercer, R.L. Class-Based n-gram Models of Natural Language. *Computational Linguistics* **1992**, *18*, 467–480.
48. Morin, F.; Bengio, Y. Hierarchical Probabilistic Neural Network Language Model. Proceedings of the International Workshop on Artificial Intelligence and Statistics (AISTATS), 2005, pp. 246–252.

Appendix A Full DFA construction

This appendix gives the deterministic finite automaton that recognizes the regular grammar of Section 3.2.4. The automaton $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ has five states $Q = \{q_0, q_1, q_2, q_3, q_4\}$, start state q_0, and accepting state $F = \{q_4\}$. Table A1 gives the state semantics.

Table A1. DFA States and Semantics

State	Meaning	Expects next
q_0	Start	`BOS`
q_1	After BOS	Command, Address, or `EOS`
q_2	After Cmd	Address or `EOS`
q_3	After Addr	`NUM` only
q_4	Accept	—

The transition function δ has seven non-trivial entries:

$$
\begin{aligned}
&\delta(q_0, \texttt{BOS}) = q_1 && \text{(begin line)}\\
&\delta(q_1, c) = q_2,\ c \in \{\texttt{G0} \ldots \texttt{G3}\} && \text{(command)}\\
&\delta(q_1, a) = q_3,\ a \in \mathit{Addr} && \text{(modal: no cmd)}\\
&\delta(q_2, a) = q_3,\ a \in \mathit{Addr} && \text{(parameter)}\\
&\delta(q_3, \texttt{NUM}) = q_2 && \text{(value} \rightarrow \text{next param)}\\
&\delta(q_1, \texttt{EOS}) = q_4 && \text{(empty block)}\\
&\delta(q_2, \texttt{EOS}) = q_4 && \text{(end after cmd/value)}
\end{aligned}
$$

Soundness.

Every string accepted by $\mathcal{A}$ lies in $L(\mathcal{G})$: state q_3 forces every address to be followed by exactly one `NUM`; commands (via $q_1 \rightarrow q_2$) are optional but must precede parameters when present; and no other orderings are reachable.

Completeness.

Every derivation in $\mathcal{G}$ maps to a unique accepting path in $\mathcal{A}$.

Cross-line modal state.

Per-line parsing is context-free of modal state. Cross-line behavior–tracking which motion command (G0–G3) is active across blocks–is modeled as a finite-state transducer $\mathcal{T}$ with five states (one per command plus an initial state). The input is a sequence of parsed blocks; the output annotates each implicit-command block with its resolved command. The composition $\mathcal{T} \circ \mathcal{A}$ remains a finite-state machine, so the full-program language is regular. Our implementation passes the modal state as an input to the per-line decoder rather than composing the automata explicitly.

State count under extended modal groups.

The 5-state DFA here covers only the motion modal group. RS-274D defines at least eight modal groups commonly set once in the preamble and held across the program: motion (G0–G3, plus the optional G38 probes and G80 cancel), plane selection (G17–G19), distance mode (G90/G91), units (G20/G21), work coordinate system (G54–G59.3), feed-rate mode (G93–G95), cutter compensation (G40–G42), and tool length offset (G43/G49). A full-coverage finite-state controller for implicit-modal persistence is therefore the Cartesian product of the per-group automata. Using the group cardinalities above, the worst-case product state count is $|G_{\text{motion}}| \cdot |G_{\text{plane}}| \cdot |G_{\text{distance}}| \cdot |G_{\text{units}}| \cdot |G_{\text{wcs}}| \cdot |G_{\text{feed}}| \cdot |G_{\text{cutter}}| \cdot |G_{\text{tlo}}| \approx 6 \cdot 3 \cdot 2 \cdot 2 \cdot 6 \cdot 3 \cdot 3 \cdot 2 = 7{,}776$ states. This remains finite and the per-step $O(1)$ mask lookup is still theoretically valid, but the flat mask table becomes impractical to build by hand; a production implementation would compile the transducer from the per-group automata (standard product construction) and cache transitions lazily. In the current work this does not arise because the preamble fixes all non-motion modal groups before the first toolpath line, leaving only the motion group to track dynamically.

Appendix B Reproducibility

We document the dataset, training pipeline, and software environment required to reproduce every result in the paper. Code, vocabulary files, trained checkpoints, and split definitions are released alongside the paper at https://github.com/stephenseacuello/gcode_fingerprinting.

Appendix B.1 Dataset and splits

The corpus comprises 120 sensor-aligned CSV files collected on a Bantam Tools Desktop CNC mill running G-code generated by Autodesk Fusion 360 CAM. The 120 files span nine distinct G-code programs covering three operation types (face milling, pocket milling, adaptive clearing) at three feed-rate settings each, with multiple repetitions per program. The unprocessed corpus contains 62,403 toolpath lines of which 2,248 are unique after canonicalization. Each CSV row contains 110 sensor features—8 electrical channels for the spindle and three axis motors, 102 inertial channels from six MPU-9250 modules at approximately 50 Hz—plus a `gcode_string` column giving the G-code line active during the row's sensor window.

All cross-validation experiments use the same five-fold partition (the "v7" fold structure in the released code repository). Each fold contains train, validation, and test splits whose sizes vary by fold (per-fold test counts 132/113/106/108/93, 552 evaluated samples in total), and the same fold definitions are used for the published hierarchical model (Table 15), the cross-tokenizer experiments (Tables 19 and 21), and the KL distortion measurement (Table 22). Files are split by source program so that no run from a held-out program leaks into the training set, which prevents trivial within-program memorization from inflating accuracy.

Appendix B.2 Tokenizer and decoder training

Domain BPE and WordPiece are trained from scratch on each fold's training split using HuggingFace `tokenizers` version 0.22 with vocabulary cap 1000 and minimum frequency 2. The special tokens `[PAD]`, `[BOS]`, `[EOS]`, `[UNK]` are forced to ids 0–3 by ordering them first in the trainer's special-tokens list, and the assignment is verified after training. Character-level uses a fixed 32-character alphabet covering canonicalized G-code text; flat-per-value builds vocabulary lazily from whitespace-separated words; GPT-4 BPE (compact) uses tiktoken's `cl100k_base` and remaps the active subset of token ids to a corpus-local id space; the hierarchical row uses the published 712-token vocabulary at `data/gcode_vocab_712.json`, unchanged from the existing model.

Every cross-tokenizer arm in Sections 6.3 and 6.4 uses an identical `FlatVocabDecoder`: 192-dim embeddings, four transformer-decoder layers, eight attention heads, 4× FFN expansion, dropout 0.3, sinusoidal positional encoding, and sensor conditioning via cross-attention to a pre-cached

encoder memory of dimension 256. Training uses Adam (lr $= 3 \times 10^{-4}$, no weight decay), batch size 32, 80 epochs, token-level cross-entropy with PAD ignored, and gradient clipping at norm 1.0. We deliberately omit scheduled sampling, focal loss, label smoothing, and heavy digit weighting in this slim trainer; these are present in the published multi-head trainer and would confound a cross-tokenizer comparison. Best checkpoint is selected by validation loss. The sensor encoder itself is the published MM-DTAE-LSTM model and is not retrained for the cross-tokenizer experiments; we instead load the cached encoder memory tensors and operation-prediction tensors released alongside the v7 fold structure.

Appendix B.3 Hardware, software, and where each table comes from

All cross-tokenizer training runs were performed on a single NVIDIA GPU running CUDA 12.1 and PyTorch 2.5.1. The full 5-fold by 6-tokenizer C/E battery (30 trainings plus 5 evaluation passes) runs end-to-end in approximately 12 minutes of wall time, and the KL distortion measurement is a single forward pass per fold and completes in under one minute total. The published hierarchical model (the `v7_best_5fold` multi-head decoder: 8 transformer layers, 12 heads, $d_{\text{model}} = 384$, grammar-constrained, no curriculum) was trained separately at a few hours per fold (script `train_sensor_multihead.py` in the released code).

All slim-trainer experiments use random seed 42 for PyTorch and NumPy, sample at temperature 1.0 with no top-k, and report argmax-decoded outputs after EOS trimming. The BPE and WordPiece grammar-validity rate of $0.00 \pm 0.00\%$ is robust to seed choice because the failure mode is in the decode rule rather than the sampling step. Each (tokenizer, fold) arm uses a single initialization, so we confine comparative claims among the passing rows to "within fold-level noise"—the paired bootstrap in Section 6.3 finds no significant ordering among them—and do not assert a seed-stable ranking. The published v7 headline model, by contrast, was retrained under five random seeds per fold (25 runs; the `v7_best_5fold_multiseed` release) and is seed-robust: the per-fold seed standard deviation is below 1 pp for token accuracy and below 1.5 pp for sequence exact-match, and the seed-averaged 5-fold means (94.6% token, 84.5% sequence) match the single-seed headline values reported in Table 15 (94.7% / 84.4%) to within seed noise. Its fold-to-fold spread therefore reflects data-fold rather than seed variance. The software stack is Python 3.10, PyTorch 2.5.1, NumPy 1.26, pandas 2.x, HuggingFace tokenizers 0.22, and tiktoken 0.12; the pipeline depends only on the standard PyTorch ecosystem and uses no closed-source components.

For traceability, Tables 13 and 14 are from the published model release, and Table 15 reports the 5-fold test metrics of the released `v7_best_5fold` multi-head decoder checkpoints. Table 16 is generated by `run_design_space.py`, Tables 19 and 21 are generated by `eval_alt.py` and aggregated across folds by `aggregate_5fold.py`, and Table 22 is generated by `measure_kl.py`, all under `scripts/experiments/grammar_design_space/` in the released code. The full preprocess–train–evaluate–aggregate pipeline can be reproduced by running `run_5fold.sh` from the repository root.

Appendix C Full-Page Figures

© 2026 by the authors. Submitted to *Sensors* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license (http://creativecommons.org/licenses/by/4.0/).

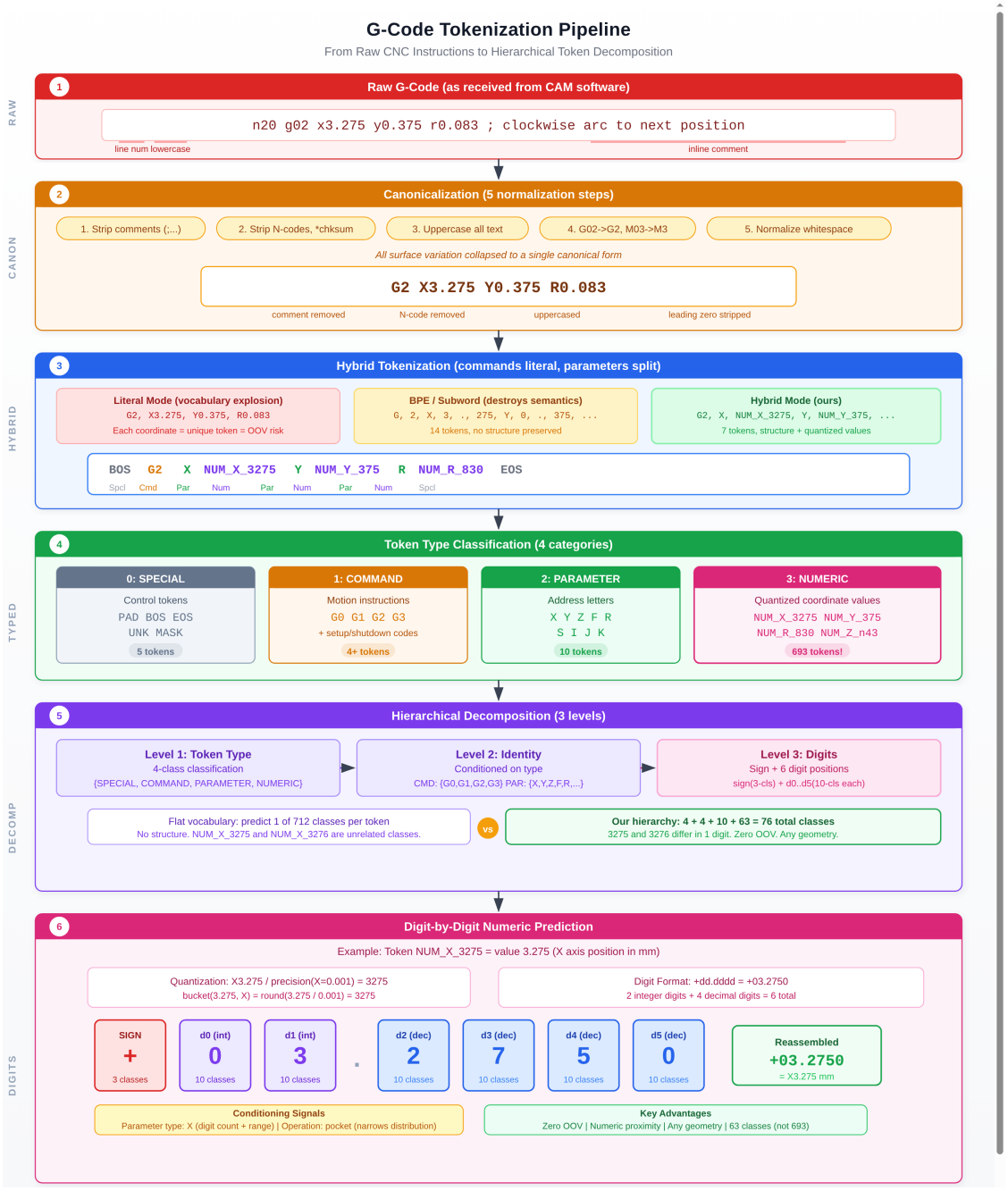

Figure A1. End-to-end tokenization pipeline. Raw G-code is canonicalized (5 steps), split into typed hybrid tokens, classified into 4 categories, decomposed hierarchically (3 levels), and predicted digit-by-digit (sign + 6 digits).

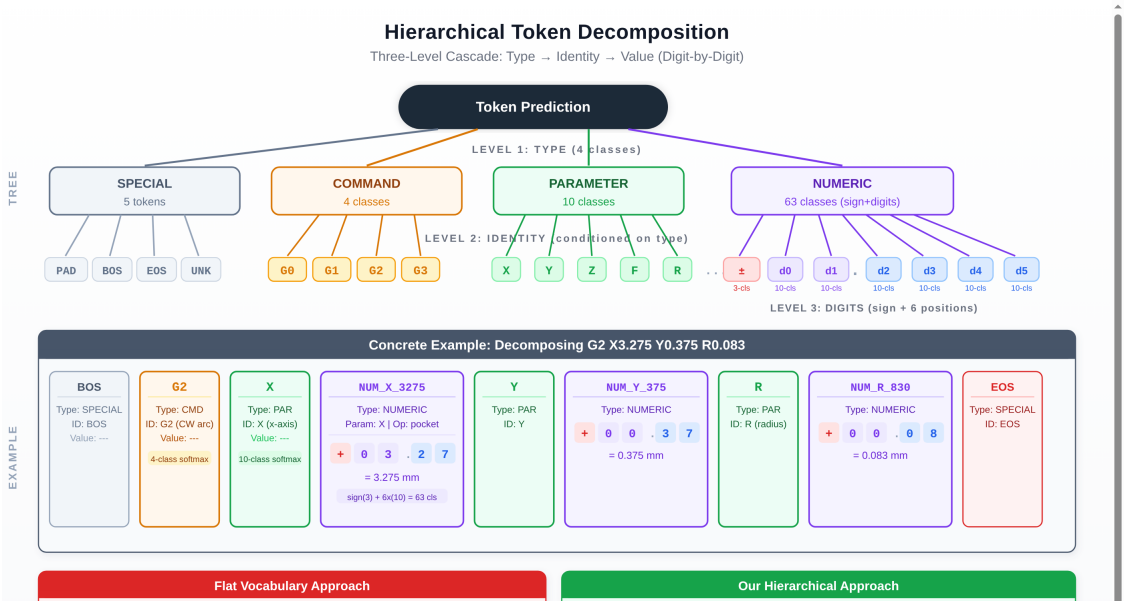

Figure A2. Hierarchical token decomposition. Top: three-level cascade (type → identity → digits) with cardinality at each branch. Bottom: concrete decomposition of `G2 X3.275 Y0.375 R0.083` showing type, identity, and digit predictions for each token.